

Memoria del proyecto de clustering

Segmentación de clientes en e-commerce

Módulo MP6 — Proyecto de IA y Big Data
Máster en IA y Big Data

Contents

Parte I — Preprocesado de datos	2
1 Introducción y contexto	2
2 Carga del dataset centrado en clientes	2
3 Resumen del preprocesado común	3
Parte II — Modelos de clustering	4
4 Planteamiento	4
5 Modelo K-Means	4
5.1 Preprocesado específico para K-Means	4
5.1.1 Transformación logarítmica	5
5.1.2 Imputación, escalado y codificación	5
5.1.3 Reducción de dimensionalidad mediante PCA	5
5.1.4 Eliminación de outliers con LOF	5
5.2 Descripción del modelo	5
5.2.1 Qué hace en este proyecto	6
5.2.2 Variables del modelo	6
5.3 Configuración del entrenamiento	7
5.4 Búsqueda del número óptimo de clusters	7
5.5 Modelo final	7
5.6 Análisis visual de los resultados	8
5.7 Consideraciones sobre la validez del modelo	8
5.7.1 Fortalezas del modelo	8
5.7.2 Limitaciones y consideraciones	9
6 Modelo Clustering Jerárquico	10
6.1 Preprocesado específico para clustering jerárquico	10
6.1.1 Detección y eliminación de outliers	10
6.1.2 Codificación, escalado y reducción de dimensionalidad	11
6.2 Descripción del modelo	11
6.3 Configuración del entrenamiento	11
6.4 Análisis del dendrograma	12
6.5 Búsqueda del número óptimo de clusters	12
6.6 Comparación de criterios de enlace	13
6.7 Modelo final y métricas	13
6.8 Análisis visual de los resultados	14

6.8.1	Visualización 2D y 3D en espacio PCA	14
6.8.2	Análisis silhouette	14
6.8.3	Heatmap de perfiles por cluster	15
6.9	Perfiles de clientes por cluster	15
6.10	Consideraciones sobre la validez del modelo	17
6.10.1	Fortalezas del modelo	17
6.10.2	Limitaciones y consideraciones	17
6.11	Visualizaciones complementarias del preprocesado (opcional)	18
6.11.1	Matriz de correlación entre variables	18
6.11.2	Varianza explicada por PCA	18
7	Modelo DBSCAN	19
7.1	Preprocesado específico para DBSCAN	19
7.1.1	Selección de variables	19
7.1.2	Escalado robusto y eliminación de outliers	19
7.1.3	Reducción de dimensionalidad	19
7.2	Descripción del modelo	19
7.3	Configuración del entrenamiento	20
7.4	Modelo final y métricas	20
7.5	Análisis visual de los resultados	20
7.5.1	Gráfico K-distancias	20
7.5.2	Visualización 2D y 3D en espacio PCA	21
7.5.3	Distribución de tamaños de clusters	21
7.5.4	Análisis Silhouette	21
7.5.5	Heatmap de perfiles por cluster	21
7.6	Perfiles de clientes por cluster	21
7.7	Consideraciones sobre la validez del modelo	23
7.7.1	Fortalezas del modelo	23
7.7.2	Limitaciones y consideraciones	23
8	Modelo HDBSCAN	24
8.1	Preprocesado específico para HDBSCAN	24
8.2	Descripción del modelo	24
8.3	Configuración del entrenamiento	25
8.4	Modelo final	25
8.5	Análisis visual de los resultados	26
8.5.1	Análisis de Silhouette	26
8.5.2	Distribución por segmento	26
8.5.3	Visualización 2D en espacio PCA	26
8.5.4	Distribución de distancias intra e inter-cluster	26
8.5.5	Matriz de distancias entre centroides	26
8.5.6	Calidad por cluster (Boxplot)	27
8.5.7	Análisis de comportamiento de compra	27
8.6	Perfiles de clientes por cluster	27
8.7	Consideraciones sobre la validez del modelo	28
8.7.1	Fortalezas del modelo	28
8.7.2	Limitaciones y consideraciones	28

9	Comparativa y modelo seleccionado	29
9.1	Tabla comparativa de los cuatro modelos	29
9.2	Modelo seleccionado: K-Means	30
9.3	Modelo de respaldo: Clustering Jerárquico	31
9.4	Modelos descartados: DBSCAN y HDBSCAN	31
9.5	Conclusión final	32

Parte I — Preprocesado de datos

1 Introducción y contexto

El objetivo de este proyecto es segmentar a los clientes de una tienda de *e-commerce* del Reino Unido. Se utiliza el mismo dataset que en el proyecto de regresión (*E-Commerce Data*, Kaggle / UCI), que recoge las transacciones realizadas entre diciembre de 2010 y diciembre de 2011. A diferencia del proyecto anterior, aquí no hay variable objetivo: el aprendizaje es no supervisado y se utiliza el dataset completo, sin separación en *train* / *test*.

La estrategia de preprocesado parte de la **limpieza realizada en la fase anterior**, pero da un paso más: convierte el dataset transaccional en un dataset **centrado en clientes**, donde cada fila representa a un único usuario con un conjunto de variables agregadas que describen su comportamiento de compra. Este nuevo dataset es el punto de partida del proceso descrito en esta parte.

Sobre el dataset de usuarios se aplica un preprocesado adicional: detección de *outliers*, codificación de variables categóricas, escalado y reducción de dimensionalidad. La limpieza de *outliers* se realiza sobre el dataset centrado en usuarios, una vez agregadas las transacciones por cliente.

2 Carga del dataset centrado en clientes

El *script* de limpieza desarrollado en la fase anterior parte del dataset transaccional original, aplica los filtros de la primera entrega (eliminación de duplicados, notas de crédito, filas sin `id_cliente` y precios no válidos) y construye un nuevo dataset a nivel de cliente. Este último paso es el más relevante para el proyecto de *clustering*: se itera sobre cada cliente único y se calculan sus variables agregadas a partir de todas sus transacciones.

```
# Generación del dataset centrado en clientes
clientes_vars = []
for cliente_id in df["id_cliente"].unique():
    cliente_data = df[df["id_cliente"] == cliente_id].copy()

    variables_cliente = {
        "id_cliente": cliente_id,
        "total_compras": len(cliente_data),
        "total_productos_diferentes": cliente_data["codigo_producto"].nunique(),
        "valor_total_compras": cliente_data["importe_linea"].sum(),
        "valor_medio_por_compra": cliente_data["importe_linea"].mean(),
        "dias_como_cliente": (cliente_data["fecha"].max()
                              - cliente_data["fecha"].min()).days + 1,
        "pais_principal": cliente_data["pais"].mode().iloc[0],
        # ... (resto de variables agregadas)
    }
    clientes_vars.append(variables_cliente)

df_clientes = pd.DataFrame(clientes_vars)
```

Por cada usuario único se calculan así 30 variables agrupadas en bloques temáticos:

- **Volumen de compras:** `total_compras`, `total_productos_diferentes`, `total_facturas`.
- **Valor monetario:** `valor_total_compras`, `valor_medio_por_compra`, `valor_mediano_por_compra`, `precio_unitario_medio`.
- **Cantidad:** `cantidad_total_productos`, `cantidad_media_por_compra`.

- **Temporales:** `primer_compra`, `ultima_compra`, `dias_como_cliente`.
- **Frecuencia y comportamiento:** `dia_semana_favorito`, `hora_favorita`, `frecuencia_compra`, `tiempo_medio_entre_compras`, `tiempo_mediano_entre_compras`.
- **Geográficas:** `pais_principal`, `países_diferentes`.
- **Comportamiento semanal:** `compras_fines_semana`, `porcentaje_compras_fines_semana`.
- **Estacionalidad:** `mes_mas_activo`, `meses_diferentes_compra`, `meses_activo`.
- **Diversidad de productos:** `categorias_productos_diferentes`, `categoria_favorita`, `diversidad_categorias`.
- **Mensuales:** `valor_mensual_medio`, `compras_mensuales_medias`.

Una variable que merece especial atención es `categoria_favorita`, ya que el dataset original no incluye una categorización explícita de los productos. Para generarla, el *script* de limpieza clasifica cada transacción en una categoría temática a partir de palabras clave en la descripción del producto:

```
def categorizar_producto(descripcion):
    desc_lower = str(descripcion).lower()
    if any(word in desc_lower for word in ["christmas", "xmas", "santa"]):
        return "navidad"
    elif any(word in desc_lower for word in ["heart", "love", "valentine"]):
        return "romantico"
    elif any(word in desc_lower for word in ["baby", "bab", "child"]):
        return "bebe"
    elif any(word in desc_lower for word in ["birthday", "cake", "party"]):
        return "cumpleanos"
    elif any(word in desc_lower for word in ["postage", "carriage", "manual"]):
        return "no_producto"
    else:
        return "otros"
```

Esta clasificación es relevante para el proyecto porque introduce una de las pocas variables categóricas del dataset, que más adelante se codificará mediante *One-Hot* (Sección ??).

El dataset resultante se carga directamente desde el archivo exportado por el *script* de limpieza:

```
RUTA_CSV = "dataset_clientes_clustering.csv"
df = pd.read_csv(RUTA_CSV)
print(f"Dataset cargado: {df.shape[0]} clientes, {df.shape[1]} variables")
```

→ Estado inicial del dataset de clientes: **4.339 clientes** y **30 variables**.

3 Resumen del preprocesado común

La fase de preprocesado común al grupo deja como resultado el dataset `dataset_clientes_clustering.csv`, con las siguientes características:

- **Filas (clientes):** 4.339 clientes únicos, agregados a partir de las transacciones del dataset original.
- **Columnas:** 30 variables que cubren volumen de compras, valor monetario, comportamiento temporal, geografía y diversidad de productos.

- **Variables categóricas:** `pais_principal` y `categoria_favorita`.
- **Variables numéricas:** las 28 restantes.

Este dataset es el punto de partida común para los tres modelos de *clustering* descritos en la Parte II (Jerárquico, K-Means y DBSCAN). Cada modelo aplicará a continuación su propio preprocesado específico, en función de las particularidades del algoritmo y de las variables que mejor exploten su lógica interna.

Parte II — Modelos de clustering

4 Planteamiento

A diferencia del proyecto de regresión, este es un problema de aprendizaje **no supervisado**: no existe variable objetivo y no es posible separar el dataset en *train* y *test*. Todo el dataset preprocesado se utiliza para entrenar cada modelo. La evaluación se basa en métricas internas de calidad del *clustering* y, sobre todo, en la **interpretabilidad de los grupos obtenidos**, que es el criterio de éxito real desde el punto de vista de negocio.

En esta parte se documentan cuatro modelos de *clustering*, cada uno con su propio preprocesado específico aplicado sobre el dataset común generado en la Parte I:

- **K-Means** (Sección 5): algoritmo basado en centroides con un número de *clusters* fijado a priori.
- **Clustering Jerárquico** (Sección 6): algoritmo aglomerativo basado en un dendrograma de fusiones sucesivas.
- **DBSCAN** (Sección 7): algoritmo basado en densidad, capaz de detectar *clusters* de forma arbitraria y de identificar puntos atípicos.
- **HDBSCAN** (Sección 8): variante jerárquica de DBSCAN que detecta automáticamente *clusters* de densidades variables.

Las métricas utilizadas son fundamentalmente tres: el **índice de silueta** (rango $[-1, 1]$, mayor mejor), el **índice de Davies-Bouldin** (rango $[0, \infty)$, menor mejor) y el **índice de Calinski-Harabasz** (mayor mejor). Algunos modelos incorporan métricas adicionales (porcentaje de ruido, equilibrio de *clusters*) cuando son relevantes para su lógica interna. Adicionalmente se inspeccionan los tamaños de los *clusters*, ya que una segmentación con un grupo que concentre la práctica totalidad de los clientes no aporta valor de negocio, por mucho que sus métricas sean buenas.

5 Modelo K-Means

5.1 Preprocesado específico para K-Means

Sobre el dataset común generado en la Parte I (4.339 clientes, 30 variables) se aplica un preprocesado específico para adaptarlo a las particularidades del algoritmo K-Means, que es sensible a valores atípicos y depende fuertemente del escalado de variables. Los pasos son: transformación logarítmica, imputación de valores faltantes, escalado y codificación, eliminación de *outliers*, y reducción de dimensionalidad mediante PCA.

5.1.1 Transformación logarítmica

Muchas variables numéricas presentan distribuciones muy asimétricas con valores extremos elevados. Para reducir esta asimetría se aplica una transformación $x' = \log(1+x)$ a las principales variables de comportamiento (compras, valor, cantidad, frecuencia, recencia). Esta transformación reduce el impacto de clientes extremadamente grandes y mejora la estabilidad del clustering.

5.1.2 Imputación, escalado y codificación

Las variables numéricas se escalan con **RobustScaler**, menos sensible a valores extremos que **StandardScaler**, lo que es esencial porque K-Means utiliza distancias euclidianas. Las variables categóricas (**pais_principal**, **categoria_favorita**, **dia_semana_favorito**, **hora_favorita**) se transforman mediante **OneHotEncoder**.

```
preprocesador = ColumnTransformer([
    ("num", RobustScaler(), columnas_numericas),
    ("cat", OneHotEncoder(handle_unknown="ignore"), columnas_categoricas),
])

X = preprocesador.fit_transform(df)
```

5.1.3 Reducción de dimensionalidad mediante PCA

Tras el escalado y codificación se aplica PCA con **10 componentes principales**, que conservan el **91,8 % de la varianza** total del dataset. Esta reducción cumple un doble papel: por un lado, elimina la redundancia entre variables y mejora la separabilidad de los *clusters* en el espacio reducido; por otro, hace que la posterior detección de *outliers* y el clustering operen sobre un espacio compacto y más informativo.

```
pca = PCA(n_components=10, random_state=42)
X_pca = pca.fit_transform(X.toarray() if hasattr(X, "toarray") else X)
```

5.1.4 Eliminación de outliers con LOF

Sobre el espacio PCA reducido se aplica *Local Outlier Factor* (LOF), un detector de anomalías basado en densidad local. Compara la densidad de cada punto con la de sus vecinos más cercanos y marca como atípicos aquellos cuya densidad local es notablemente menor. Se configura con **n_neighbors=20** y **contamination=0.03** (3 % de los clientes esperados como atípicos).

```
from sklearn.neighbors import LocalOutlierFactor

lof = LocalOutlierFactor(n_neighbors=20, contamination=0.03)
labels_lof = lof.fit_predict(X_pca)
mask = labels_lof == -1
X_modelo = X_pca[mask]
df_modelo = df[mask].copy()
```

→ Tras la eliminación de *outliers*, el dataset queda con **4.208 clientes** (un 3 % eliminado del dataset original de 4.339), igualmente listos para el *clustering* sobre el espacio PCA de 10 dimensiones.

5.2 Descripción del modelo

K-Means es un algoritmo de aprendizaje no supervisado utilizado para realizar tareas de *clustering* o segmentación de datos. Su objetivo principal es dividir un conjunto de observaciones en un número determinado de grupos (*clusters*), de forma que los elementos dentro de cada grupo sean

lo más similares posible entre sí y lo más diferentes posible respecto a los elementos de otros grupos.

El algoritmo funciona mediante un proceso iterativo basado en distancias. Inicialmente selecciona de manera aleatoria un conjunto de centroides, uno por cada *cluster* definido. Posteriormente, cada punto del dataset se asigna al centroide más cercano utilizando normalmente la distancia euclidiana. Una vez asignados todos los puntos, el algoritmo recalcula los centroides como la media de todos los puntos pertenecientes a cada *cluster*. Este proceso se repite hasta alcanzar la convergencia, es decir, hasta que las asignaciones dejan de cambiar o las variaciones son mínimas.

K-Means es uno de los algoritmos de *clustering* más utilizados debido a su simplicidad, rapidez y facilidad de interpretación. Sin embargo, presenta algunas limitaciones importantes: requiere definir previamente el número de *clusters* k , es sensible a los valores atípicos y asume que los grupos tienen una forma aproximadamente esférica y tamaños similares.

En este proyecto, el algoritmo se complementa con un proceso previo de detección y eliminación de *outliers* mediante *Local Outlier Factor* (LOF), reduciendo así el impacto negativo de clientes extremadamente atípicos sobre los centroides y mejorando la estabilidad de la segmentación.

5.2.1 Qué hace en este proyecto

K-Means se aplica a la segmentación de clientes con el objetivo de identificar grupos homogéneos de comportamiento que permitan personalizar estrategias de *marketing*, promociones y acciones comerciales. El *pipeline* realiza varias etapas consecutivas:

- Preparación y transformación de variables (transformación logarítmica de variables asimétricas).
- Escalado de variables numéricas y codificación de categóricas.
- Reducción de dimensionalidad mediante PCA de 10 componentes para mejorar la estabilidad del *clustering* y reducir el ruido dimensional.
- Detección y eliminación de *outliers* mediante LOF.
- Búsqueda automática del número óptimo de *clusters* usando métricas internas de calidad.
- Entrenamiento del modelo final K-Means.
- Reducción de dimensionalidad a 2 componentes exclusivamente para visualización gráfica.
- Generación de métricas, visualizaciones y resúmenes estadísticos de cada segmento.
- Exportación de los resultados a un fichero CSV para su integración en herramientas de CRM o *marketing*.

5.2.2 Variables del modelo

El modelo utiliza variables tanto numéricas como categóricas relacionadas con el comportamiento de compra del cliente.

Variables numéricas (transformadas con $\log(1 + x)$ las indicadas):

- `total_compras_log`, `total_productos_diferentes_log`, `total_facturas_log`: volumen y diversidad de la actividad.
- `valor_total_compras_log`, `valor_medio_por_compra_log`, `precio_unitario_medio_log`: comportamiento monetario.
- `cantidad_total_productos_log`, `cantidad_media_por_compra_log`: volumen físico de productos.

- `dias_como_cliente_log`, `recencia_dias_log`: antigüedad y recencia.
- `países_diferentes`, `categorías_productos_diferentes`, `diversidad_categorías`: diversidad geográfica y de productos.
- `compras_fines_semana_log`, `porcentaje_compras_fines_semana`: patrón temporal semanal.
- `mes_mas_activo`, `meses_diferentes_compra`, `meses_activo_log`: patrones estacionales.
- `tiempo_medio_entre_compras_log`, `frecuencia_compra_log`: cadencia de compra.
- `valor_mensual_medio_log`, `compras_mensuales_medias_log`: actividad mensual.

Variables categóricas (codificadas con *One-Hot Encoding*):

- `pais_principal`: país principal del cliente.
- `categoría_favorita`: categoría de producto más comprada.
- `día_semana_favorito`: día habitual de compra.
- `hora_favorita`: franja horaria más frecuente de compra.

5.3 Configuración del entrenamiento

Se utiliza la implementación `KMeans` de *scikit-learn* con `n_init=30` durante la búsqueda y `n_init=50` en el modelo final, lo que reduce la probabilidad de converger a mínimos locales subóptimos. La estrategia de selección del k óptimo consiste en entrenar el modelo para $k \in \{2, 3, 4, 5, 6, 7, 8, 9\}$ y evaluar cada configuración mediante el **índice de silueta** como métrica principal, complementado con análisis visual del **índice de Calinski-Harabasz** y el **índice de Davies-Bouldin**.

```
resultados = []
for k in range(2, 10):
    modelo = KMeans(n_clusters=k, random_state=42, n_init=30)
    etiquetas = modelo.fit_predict(X_modelo)
    sil = silhouette_score(X_modelo, etiquetas)
    resultados.append((k, sil))

mejor_k = sorted(resultados, key=lambda x: x[1], reverse=True)[0][0]
```

5.4 Búsqueda del número óptimo de clusters

La Figura 1 muestra la evolución de las tres métricas en función de k . La silueta presenta un máximo claro en $k = 2$ (0,5257) con una caída pronunciada a 0,2627 en $k = 3$. Los índices complementarios refuerzan esta elección: Calinski-Harabasz alcanza su máximo en $k = 2$ y desciende monótonamente a partir de ahí, mientras que Davies-Bouldin presenta su mínimo en $k = 2$ y empeora con valores mayores. Los tres indicadores convergen en señalar $k = 2$ como configuración óptima.

5.5 Modelo final

La configuración seleccionada es `KMeans(n_clusters=2, n_init=50, random_state=42)` entrenada sobre el dataset preprocesado de 4.208 clientes en el espacio PCA de 10 dimensiones. Los tamaños de los dos *clusters* resultantes son 3.673 y 535 clientes (Figura 2). El Cuadro 2 resume las métricas finales.

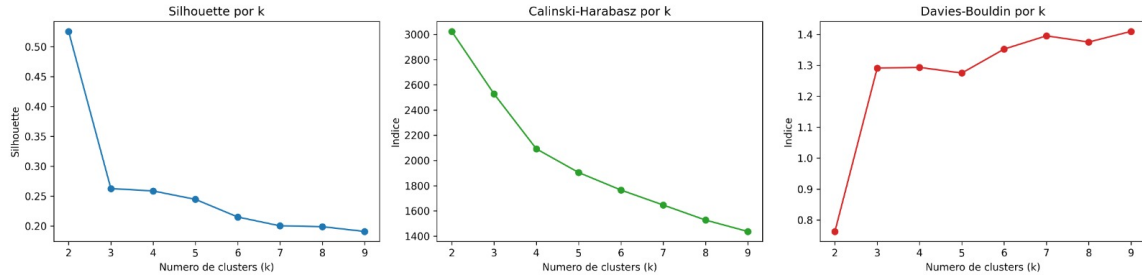


Figure 1: Evolución de las tres métricas internas en función del número de *clusters* k : silueta (mayor mejor), Calinski-Harabasz (mayor mejor) y Davies-Bouldin (menor mejor).

Table 1: Índice de silueta para distintos valores de k

k	Silueta
2	0,5257
3	0,2627
4	0,2587
5	0,2446
6	0,2151
7	0,2003
8	0,1989
9	0,1909

5.6 Análisis visual de los resultados

La Figura 3 muestra los *clusters* proyectados en el espacio PCA 2D. Se aprecia una separación nítida entre el *cluster* 0 (mayoritario, en azul, concentrado en la zona izquierda del eje PC1) y el *cluster* 1 (minoritario, en naranja, distribuido hacia valores altos de PC1). La separación es coherente con el valor de silueta obtenido (0,5257).

5.7 Consideraciones sobre la validez del modelo

5.7.1 Fortalezas del modelo

- **Silueta sólida:** 0,5257, dentro del rango “buena estructura” ($>0,5$), con una caída pronunciada y limpia entre $k = 2$ y $k = 3$ (de 0,5257 a 0,2627) que confirma la elección.
- **Convergencia entre criterios:** las tres métricas internas (silueta, Calinski-Harabasz y Davies-Bouldin) señalan de forma unánime $k = 2$ como configuración óptima.
- **Preprocesado robusto:** la combinación de PCA (10 componentes, 91,8 % de varianza) y *LOF* sobre el espacio reducido produce una representación compacta del comportamiento de compra y elimina anomalías locales antes del *clustering*.
- **Interpretabilidad de negocio:** la naturaleza basada en centroides permite caracterizar cada *cluster* con un punto representativo, facilitando que los resultados sean comprendidos tanto por perfiles técnicos como por responsables de *marketing*.
- **Aplicabilidad directa en CRM:** la asignación de cada cliente a un segmento concreto se exporta a CSV y es directamente integrable con plataformas de automatización comercial.
- **Rapidez y estabilidad:** el algoritmo es eficiente incluso en datasets grandes y el uso de `n_init=50` reduce la dependencia de la inicialización aleatoria.

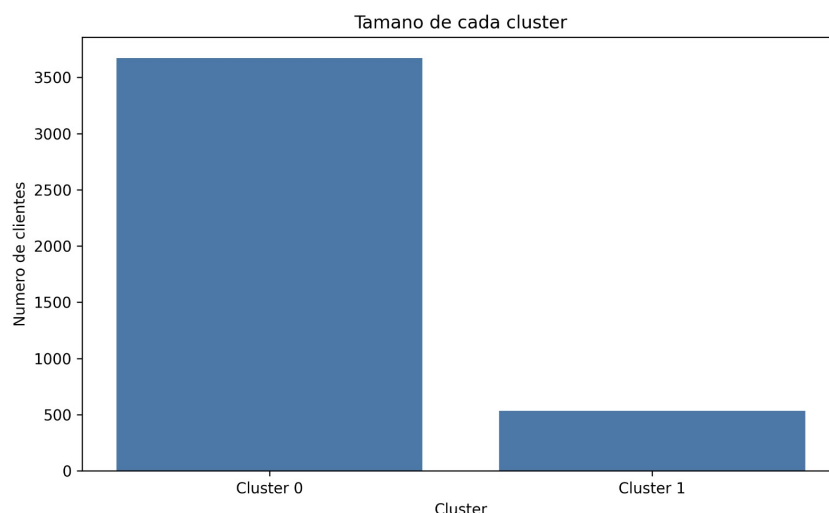


Figure 2: Distribución del número de clientes por *cluster*.

Table 2: Resumen del modelo K-Means final

Métrica / parámetro		Valor
Algoritmo		KMeans
k (número de <i>clusters</i>)		2
<code>n_init</code>		50
<code>random_state</code>		42
Componentes PCA	10 (91,8 % varianza)	
Clientes utilizados		4.208
Silueta		0,5257
Tamaño <i>cluster</i> 0		3.673 (87,3 %)
Tamaño <i>cluster</i> 1		535 (12,7 %)

5.7.2 Limitaciones y consideraciones

- **Necesidad de fijar k a priori:** aunque la búsqueda automática mitiga este problema, obliga a realizar procesos de validación adicionales para seleccionar el valor óptimo.
- **Asunción de clusters esféricos:** K-Means funciona mejor con grupos de forma aproximadamente convexa y tamaños similares; en este caso los tamaños son desbalanceados (87,3 % / 12,7 %).
- **Pérdida de interpretabilidad directa por PCA:** la reducción de dimensionalidad mejora la estabilidad del modelo, pero hace que el *clustering* se realice sobre componentes transformados y no sobre las variables originales, lo que dificulta atribuir un *cluster* a una variable concreta sin un análisis adicional.
- **Sensibilidad a valores extremos:** aunque la transformación logarítmica y la eliminación de *outliers* mediante LOF mitigan este problema, el algoritmo sigue siendo sensible a valores atípicos residuales.
- **Dependencia del escalado:** cualquier diferencia de escala entre variables se traduce directamente en mayor peso en el cálculo de distancias.
- **Sin detección de anomalías propia:** a diferencia de los modelos basados en densidad, K-Means asigna obligatoriamente cada cliente a un *cluster*, por lo que la identificación de *outliers* debe hacerse en una etapa previa.

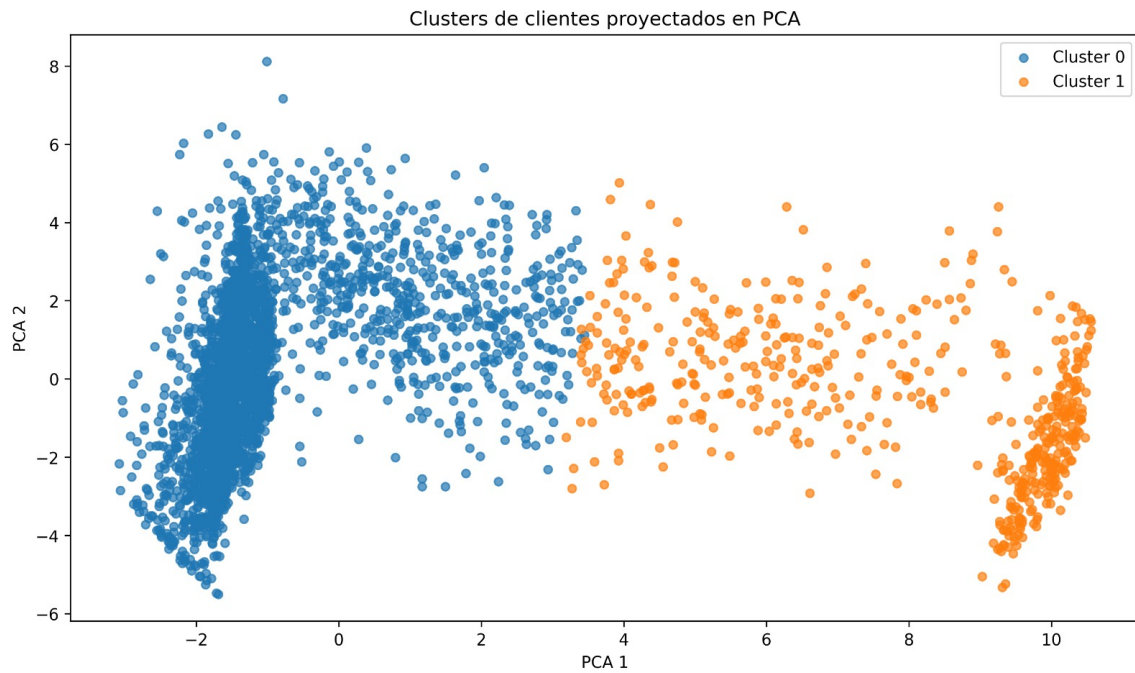


Figure 3: Clientes proyectados sobre las dos primeras componentes principales, coloreados por *cluster*.

- **Pequeñas variaciones en los datos pueden modificar centroides:** como ocurre en cualquier algoritmo de *clustering*, los resultados deben interpretarse como una aproximación útil para segmentación comercial y no como categorías absolutas e inmutables.

6 Modelo Clustering Jerárquico

6.1 Preprocesado específico para clustering jerárquico

Sobre el dataset común generado en la Parte I (4.339 clientes, 30 variables) se aplica un preprocesado específico para adaptarlo al algoritmo jerárquico, que se basa en distancias entre puntos. Los pasos son: detección y eliminación de *outliers*, codificación de variables categóricas, escalado y reducción de dimensionalidad mediante PCA.

6.1.1 Detección y eliminación de outliers

Una inspección preliminar revela la presencia de clientes “ballena”: un cliente con `valor_total_compras` superior a 77.000 GBP (frente a una mediana en torno a los 660 GBP) y otros con cantidades de producto desproporcionadas respecto al resto. Estos casos extremos distorsionan el espacio de características y se aplica una estrategia en dos pasos:

- **Paso 1 — Recorte por percentil 99** en variables monetarias y de cantidad (`valor_total_compras`, `cantidad_total_productos`, `total_compras`, `valor_medio_por_compra`, `valor_mensual_medio`, `compras_mensuales_medias` y `cantidad_media_por_compra`). Elimina 161 clientes (3,71 %).
- **Paso 2 — Isolation Forest** sobre el dataset ya recortado, configurado con `contamination=0.05` y `n_estimators=200`. Detecta 209 *outliers* multivariantes adicionales (5 %).

```
from sklearn.ensemble import IsolationForest
```

```
iso_forest = IsolationForest(contamination=0.05, random_state=42, n_estimators=200)
outlier_labels = iso_forest.fit_predict(df[num_cols_outliers])
df = df[outlier_labels == 1].reset_index(drop=True)
```

→ El dataset final tras los dos pasos contiene **3.969 clientes**, una pérdida total del 8,53 % respecto al original.

6.1.2 Codificación, escalado y reducción de dimensionalidad

El dataset contiene dos variables categóricas: `pais_principal` (36 valores únicos) y `categoria_favorita` (6). Para evitar generar columnas con muy poca señal, se agrupan en una categoría `Other` todas las categorías con frecuencia inferior al 1 %. Tras la agrupación, ambas variables quedan reducidas a 4 categorías cada una y se aplica codificación *One-Hot*, resultando en 33 columnas.

A continuación se aplica `RobustScaler`, que centra cada variable en su mediana y la escala por el rango intercuartílico (IQR). Frente a `StandardScaler`, es menos sensible a *outliers* residuales que pudieran haber sobrevivido a la depuración anterior.

Finalmente, el análisis de la matriz de correlaciones detecta **19 pares con $|r| > 0,7$** , evidenciando una redundancia importante en variables derivadas del mismo conjunto de transacciones. Se aplica PCA seleccionando el número mínimo de componentes que explican al menos el 80 % de la varianza, con un mínimo de 3. Con **3 componentes principales** se cubre el **84,14 % de la varianza**, y el dataset resultante $X_{PCA} \in \mathbb{R}^{3969 \times 3}$ constituye la entrada del modelo.

Table 3: Reducción de filas en el preprocesado del clustering jerárquico

Etapa	Eliminadas	Restantes	% conservado
Dataset común (Parte I)	—	4.339	100,00 %
Paso 1 — Recorte percentil 99	161	4.178	96,29 %
Paso 2 — <i>Isolation Forest</i> (5 %)	209	3.969	91,47 %

6.2 Descripción del modelo

El *clustering* jerárquico aglomerativo es un algoritmo de aprendizaje no supervisado que construye una jerarquía de agrupaciones de forma ascendente: parte de cada punto como su propio *cluster* y, en cada iteración, fusiona los dos más cercanos según un criterio de enlace (*linkage*). El proceso continúa hasta que todos los puntos quedan en un único *cluster*, generando un árbol de fusiones llamado **dendrograma**.

A diferencia de *K-Means*, no requiere fijar el número de *clusters* de antemano: la decisión sobre cuántos grupos extraer se toma a posteriori, “cortando” el dendrograma a la altura adecuada. Esto convierte al dendrograma en una herramienta de diagnóstico única de este algoritmo: los saltos grandes en la altura de fusión indican configuraciones donde los *clusters* están bien separados.

El criterio de enlace determina cómo se mide la distancia entre dos *clusters*. Las opciones principales son *ward* (minimiza la varianza intra-*cluster*, tiende a generar *clusters* compactos y de tamaño similar), *complete* (distancia máxima entre puntos), *average* (distancia media) y *single* (distancia mínima, suele formar cadenas alargadas y es muy sensible a *outliers*).

6.3 Configuración del entrenamiento

Se utiliza la implementación `AgglomerativeClustering` de *scikit-learn*, complementada con la función `linkage` de *SciPy* para calcular y visualizar el dendrograma. La distancia es euclídea y, salvo en la comparación de criterios, el enlace es *ward*.

```

from sklearn.cluster import AgglomerativeClustering
from scipy.cluster.hierarchy import linkage, dendrogram

Z = linkage(X_pca, method='ward')
modelo = AgglomerativeClustering(n_clusters=K, linkage='ward')
labels = modelo.fit_predict(X_pca)

```

La estrategia de evaluación tiene tres fases: (1) inspección del dendrograma para identificar visualmente posibles puntos de corte; (2) búsqueda sistemática del número óptimo de *clusters* probando $K \in \{2, 3, \dots, 10\}$ y comparando silueta, Davies-Bouldin e inercia intra-*cluster*; (3) fijado el K óptimo, comparación de los cuatro criterios de enlace.

6.4 Análisis del dendrograma

El dendrograma sobre los 3.969 clientes en el espacio PCA, calculado con enlace *ward*, se muestra en la Figura 4. Por legibilidad se ha truncado a las últimas 30 fusiones (modo `truncate_mode='lastp'`); el árbol completo es ilegible con este tamaño de muestra.

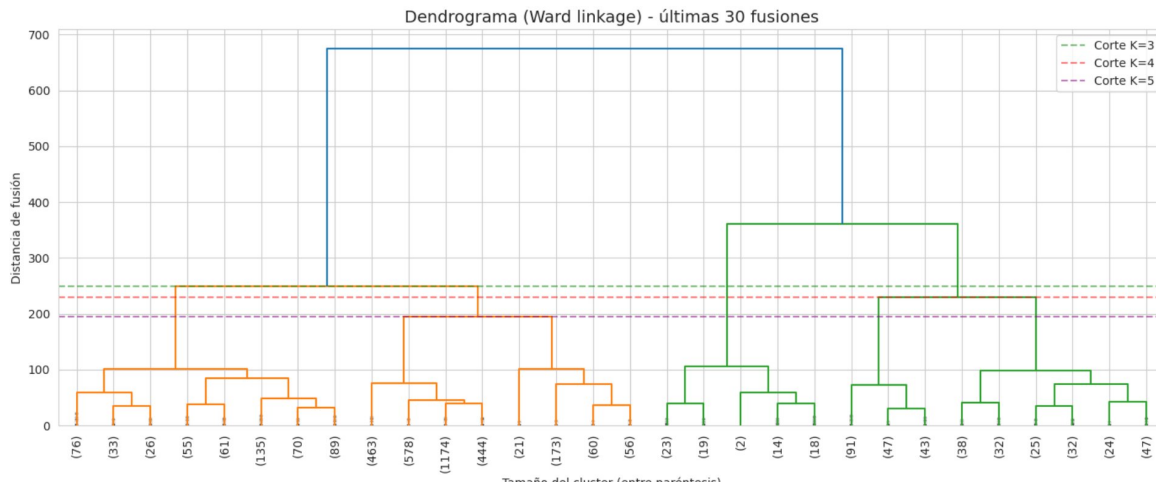


Figure 4: Dendrograma del *clustering* jerárquico (enlace *ward*, últimas 30 fusiones). Las líneas discontinuas marcan los cortes correspondientes a $K = 3$ (verde), $K = 4$ (rojo) y $K = 5$ (morado).

El salto más pronunciado en la altura de fusión se produce entre las dos últimas uniones, lo que sugiere visualmente que la separación más nítida de la estructura del dataset corresponde a $K = 2$. Para valores de K mayores los saltos son progresivamente menores, indicando que las fusiones siguientes ya no separan grupos cualitativamente distintos.

6.5 Búsqueda del número óptimo de clusters

Se entrena el modelo con *ward* para todos los valores $K \in \{2, \dots, 10\}$ y se calculan las métricas para cada uno. Los resultados se recogen en el Cuadro 4 y se representan en la Figura 5.

La silueta máxima se alcanza en $K = 2$ (0,7403), con una diferencia notable respecto a los siguientes valores. El Davies-Bouldin muestra su mínimo en $K = 3$ (0,6517), pero a costa de fragmentar uno de los grupos en dos subgrupos muy desbalanceados (76 y 3.514 clientes). Se selecciona $K = 2$ como configuración final por la mayor separación en silueta y por presentar tamaños más equilibrados (455 y 3.514).

Table 4: Métricas de evaluación del *clustering* jerárquico (enlace *ward*) para distintos valores de K

K	Silueta	Davies-Bouldin	Inercia	Tam. mín.	Tam. máx.
2	0,7403	0,7186	201.949	455	3.514
3	0,7184	0,6517	136.578	76	3.514
4	0,5749	0,9455	105.398	76	2.969
5	0,5919	0,7779	78.884	76	2.969
6	0,5972	0,7685	59.884	76	2.659
7	0,5931	0,7895	54.321	34	2.659
8	0,5796	0,7151	49.154	21	2.659
9	0,5803	0,8401	44.028	21	2.659
10	0,5806	0,8879	39.274	21	2.659

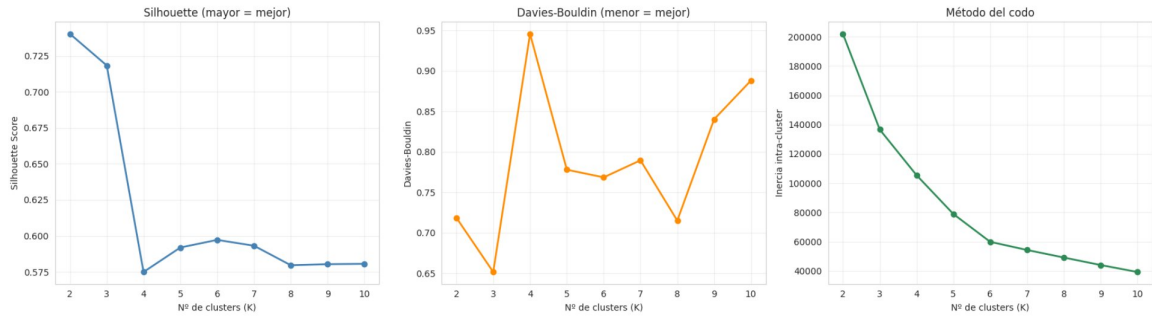


Figure 5: Evolución de las métricas de calidad del *clustering* en función de K : silueta (izquierda, mayor mejor), Davies-Bouldin (centro, menor mejor) e inercia intra-*cluster* (derecha, método del codo).

6.6 Comparación de criterios de enlace

Para confirmar la elección de *ward*, se comparan los cuatro criterios de enlace con $K = 2$ fijo. Los resultados aparecen en el Cuadro 5.

Table 5: Comparación de criterios de enlace ($K = 2$)

Linkage	Silueta	Davies-Bouldin	Tamaños
<i>ward</i>	0,7403	0,7186	455 / 3.514
<i>complete</i>	0,8190	0,2738	36 / 3.933
<i>average</i>	0,8726	0,1197	2 / 3.967
<i>single</i>	0,8726	0,1197	2 / 3.967

A primera vista, *average* y *single* ofrecen la mejor silueta (0,87) y el mejor Davies-Bouldin (0,12). Sin embargo, ambos producen un *cluster* con 2 clientes y otro con 3.967, lo que indica que están identificando *outliers* residuales, no segmentos de mercado. *Complete* (36 / 3.933) también es desbalanceado. Solo *ward* produce *clusters* con tamaños operativamente útiles (455 y 3.514), por lo que se confirma como criterio final.

6.7 Modelo final y métricas

La configuración seleccionada es `AgglomerativeClustering(n_clusters=2, linkage='ward')` entrenada sobre los 3.969 clientes en el espacio PCA de 3 dimensiones. Las métricas finales se recogen en el Cuadro 6.

Table 6: Resumen del modelo final

Métrica / parámetro		Valor
Algoritmo	AgglomerativeClustering	
Linkage	ward	
K (número de <i>clusters</i>)	2	
Cientes utilizados	3.969	
Componentes PCA	3 (84,14 % varianza)	
Silueta	0,7403	
Davies-Bouldin	0,7186	
Tamaño <i>cluster</i> 0	455 (11,5 %)	
Tamaño <i>cluster</i> 1	3.514 (88,5 %)	

6.8 Análisis visual de los resultados

Se generan cuatro visualizaciones para diagnosticar la calidad de la segmentación: proyección en PCA 2D y 3D, análisis de silueta por *cluster* y heatmap del perfil medio.

6.8.1 Visualización 2D y 3D en espacio PCA

Las proyecciones 2D y 3D (Figuras 6 y 7) permiten comprobar visualmente si los grupos están separados en el espacio de mayor varianza. Se observa una nube principal compacta (*cluster* 1) y una distribución más dispersa que se extiende hacia la derecha y abajo (*cluster* 0).

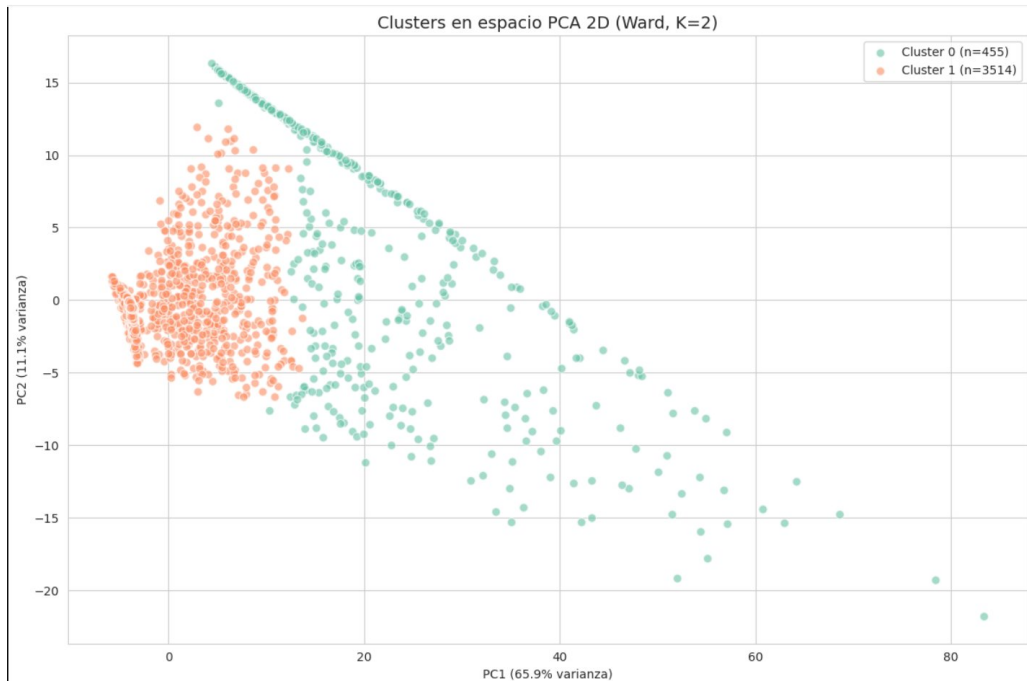


Figure 6: Clientes proyectados sobre las dos primeras componentes principales, coloreados por *cluster*.

6.8.2 Análisis silhouette

El gráfico de silueta (Figura 8) muestra el coeficiente individual de cada cliente. La línea discontinua roja indica la media general (0,740). Una buena segmentación se caracteriza por barras anchas y altas, sin valores negativos.

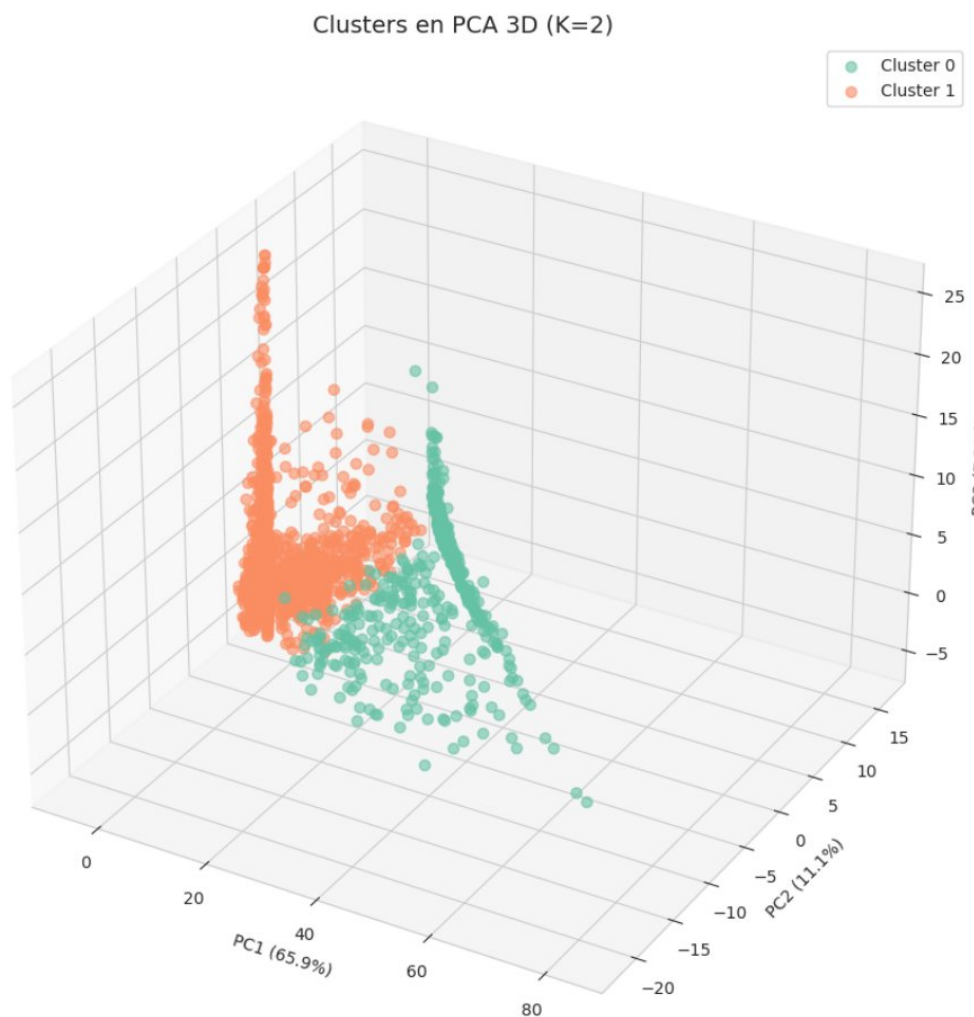


Figure 7: Clientes proyectados en las tres componentes principales del análisis PCA.

6.8.3 Heatmap de perfiles por cluster

El *heatmap* (Figura 9) visualiza las características promedio de cada *cluster* normalizadas como *z*-scores. Los colores rojos indican valores altos respecto a la media global, los azules valores bajos. Las anotaciones contienen los valores medios reales.

6.9 Perfiles de clientes por cluster

Los valores medios de las variables clave por *cluster* se presentan en el Cuadro 7. La variable que diferencia ambos grupos de forma abrumadora es **porcentaje_compras_fines_semana**: 78,64 % en el *cluster* 0 frente a 4,39 % en el *cluster* 1. El resto de variables muestran diferencias mucho menores; el valor monetario total y mensual es prácticamente idéntico entre ambos grupos.

Cluster 0 — Compradores de fin de semana (455 clientes).

- Compras en fin de semana: 78,6 % (muy alto)
- Total compras: 105,0 (alto)
- Frecuencia compra: 12,17 (alta)
- Productos diferentes: 80,0 (variedad amplia)

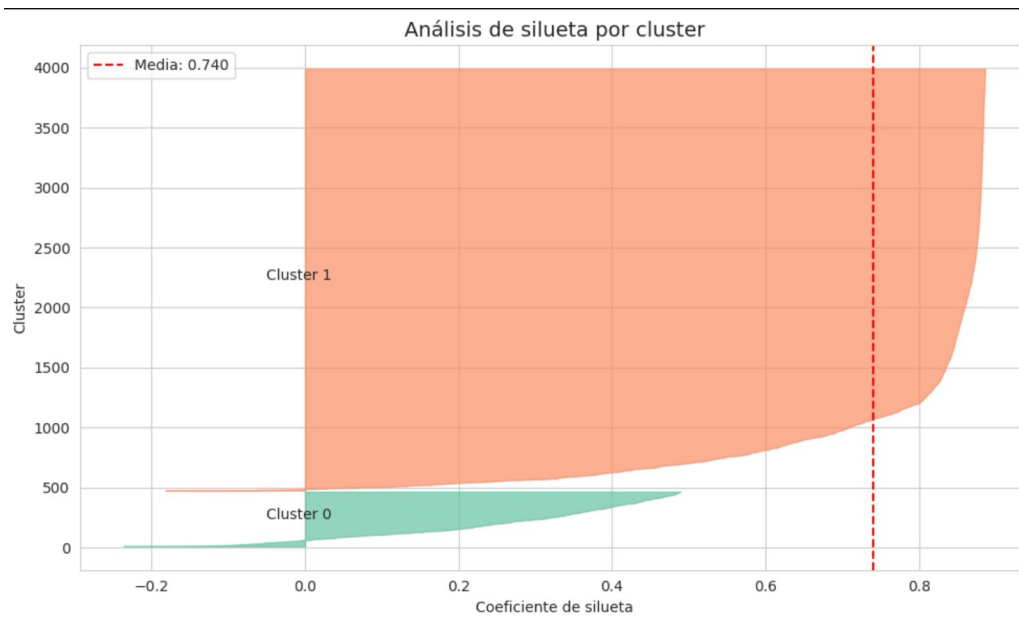


Figure 8: Coeficiente de silueta individual de cada cliente, agrupado por *cluster*.

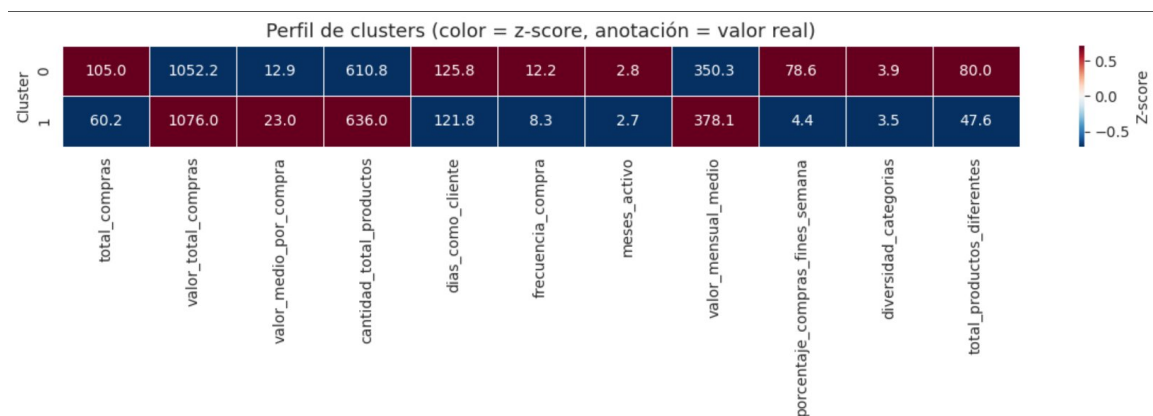


Figure 9: Perfil de *clusters*: el color codifica el *z-score* por columna; las anotaciones contienen los valores medios reales.

- **Perfil:** clientes con compras concentradas en sábado/domingo, ticket medio bajo y cesta diversificada. Compatible con clientes B2C minoristas que compran de forma recreativa fuera del horario laboral.

Cluster 1 — Compradores entre semana (3.514 clientes).

- Compras en fin de semana: 4,4 % (muy bajo)
- Valor medio por compra: 22,99 GBP (alto)
- Valor mensual medio: 378,07 GBP (alto)
- Total compras: 60,2 (medio-bajo, compensado por ticket alto)
- **Perfil:** la mayoría del dataset. Compran de lunes a viernes, con menor frecuencia pero mayor ticket medio. Compatible con clientes B2B mayoristas o profesionales que realizan pedidos en horario laboral.

Table 7: Perfil medio por *cluster*

Variable	Cluster 0 (n=455)	Cluster 1 (n=3.514)
total_compras	104,98	60,20
valor_total_compras (GBP)	1.052,20	1.076,04
valor_medio_por_compra (GBP)	12,88	22,99
cantidad_total_productos	610,83	636,04
dias_como_cliente	125,83	121,75
frecuencia_compra	12,17	8,27
valor_mensual_medio (GBP)	350,28	378,07
porcentaje_compras_fines_semana	78,64 %	4,39 %
total_productos_diferentes	79,97	47,63

6.10 Consideraciones sobre la validez del modelo

6.10.1 Fortalezas del modelo

- **Métricas internas robustas:** silueta de 0,7403, dentro del rango considerado “estructura fuerte” en la literatura ($> 0,7$); Davies-Bouldin de 0,7186, indicando buena separación.
- **Clusters de tamaño operativamente útil:** 88,5 % / 11,5 %, lejos del riesgo de tener un grupo dominante con el 99 % de los datos.
- **Diagnóstico visual único:** el dendrograma confirma que $K = 2$ corresponde al corte natural del dataset, una herramienta de la que *K-Means* no dispone.
- **Determinismo:** el algoritmo es reproducible al 100 %, sin depender de inicialización aleatoria.
- **Interpretabilidad de negocio:** los grupos se asocian a perfiles claros (B2C de fin de semana vs B2B entre semana) que permiten acciones comerciales diferenciadas.

6.10.2 Limitaciones y consideraciones

- **Separación unidimensional:** la práctica totalidad de la diferencia entre los dos grupos se concentra en una única variable (*porcentaje_compras_fines_semana*). Las demás dimensiones del comportamiento de compra son muy similares entre ambos.
- **Estructura limitada del dataset:** para $K > 2$ las métricas se degradan significativamente y los *clusters* se desbalancean, lo que sugiere que el dataset no contiene de forma natural más de dos segmentos bien diferenciados.
- **Desbalance entre grupos:** 88,5 % / 11,5 % limita el valor de las acciones específicas que se podrían dirigir al *cluster* minoritario.
- **Coste computacional:** el algoritmo es $O(n^2)$ en memoria. En este dataset (3.969 clientes) no afecta, pero sería un problema en bases de cientos de miles de clientes.
- **Juicio cualitativo en la elección final:** la decisión de K y de *linkage* incorpora consideraciones de equilibrio de tamaños, no es puramente automática a partir de las métricas.

6.11 Visualizaciones complementarias del preprocesado (opcional)

Esta sección recoge dos gráficas generadas durante el preprocesado del notebook que enriquecen visualmente el análisis pero no son imprescindibles para entender el proceso. Se incluyen como apoyo opcional y pueden eliminarse sin afectar a la coherencia del documento.

6.11.1 Matriz de correlación entre variables

La Figura 10 muestra la matriz de correlación completa entre las 33 variables del dataset escalado. Permite visualizar de un vistazo los bloques de variables redundantes: el cuadrante superior izquierdo concentra las correlaciones más fuertes (variables de volumen y monetarias), mientras que las variables categóricas codificadas (parte inferior derecha) presentan correlaciones débiles con el resto.

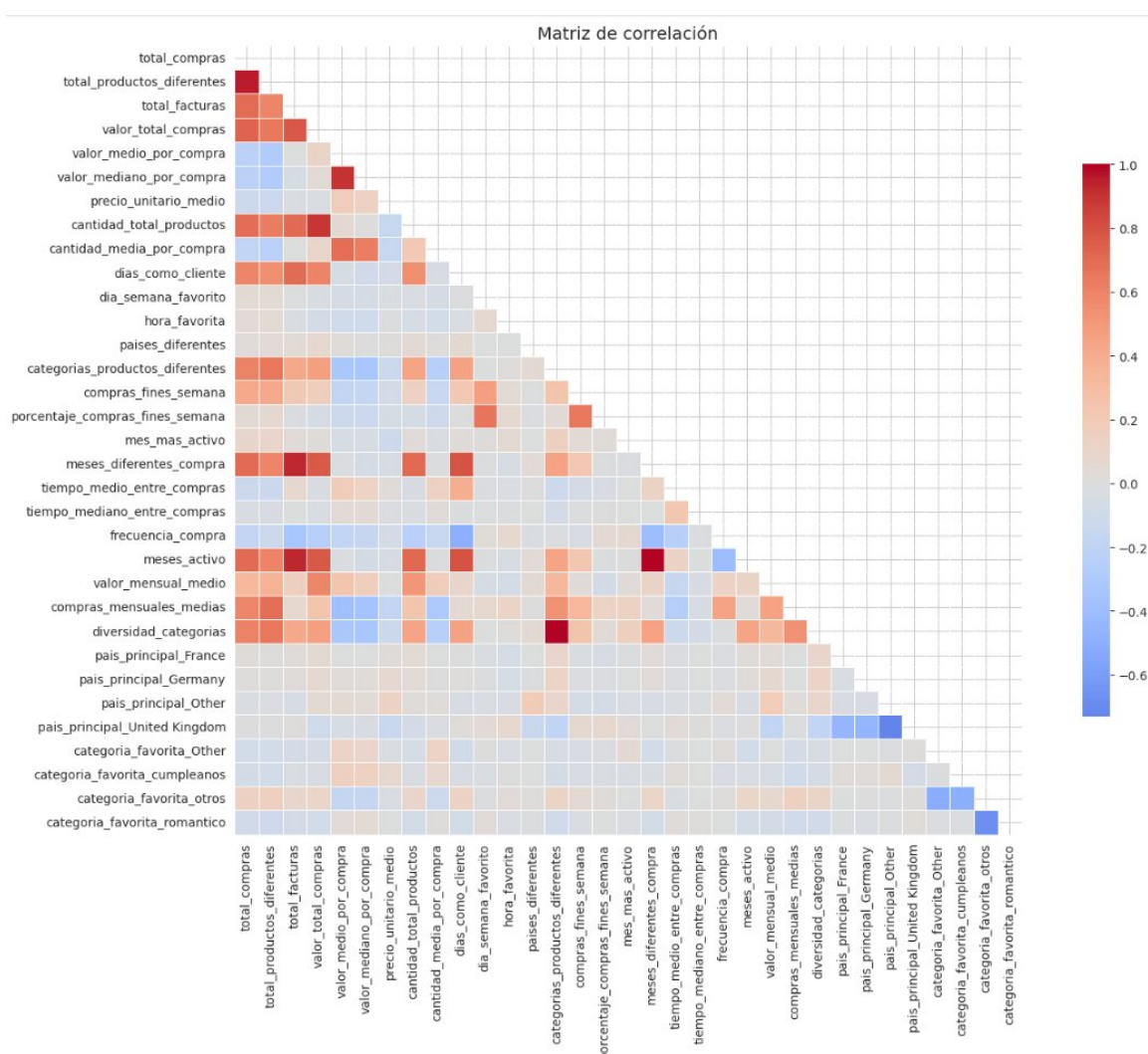


Figure 10: Matriz de correlación entre las 33 variables del dataset escalado. Los tonos rojos indican correlación positiva fuerte, los azules correlación negativa.

6.11.2 Varianza explicada por PCA

La Figura 11 muestra la varianza explicada por cada componente individual y la curva acumulada. La primera componente recoge por sí sola un 65,9 % de la varianza, la segunda un 11,1 % adicional

y la tercera un 7,2 %. A partir de la cuarta componente la contribución cae por debajo del 4 %, lo que sugiere que las tres primeras concentran la práctica totalidad de la información estructural del dataset.

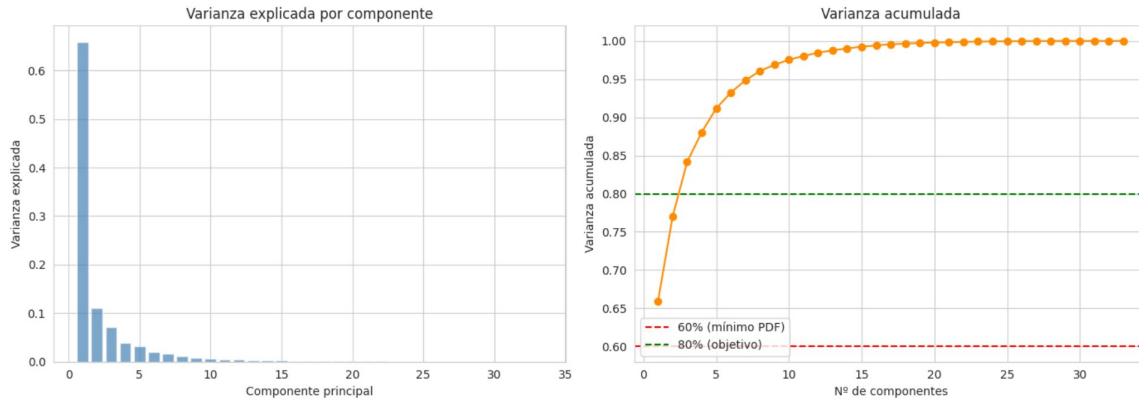


Figure 11: Análisis de varianza explicada por PCA. A la izquierda, varianza explicada por cada componente individual; a la derecha, curva acumulada con líneas de referencia en el 60 % y el 80 %.

7 Modelo DBSCAN

7.1 Preprocesado específico para DBSCAN

Sobre el dataset común generado en la Parte I se aplica un preprocesado específico que prioriza la robustez frente a valores extremos, ya que DBSCAN es sensible a la escala y a la dimensionalidad de los datos.

7.1.1 Selección de variables

Se utilizan las 9 variables más representativas del comportamiento de compra:

- `total_compras`, `valor_total_compras`, `valor_medio_por_compra`
- `frecuencia_compra`, `dias_como_cliente`, `cantidad_total_productos`
- `porcentaje_compras_fines_semana`, `meses_activo`, `valor_mensual_medio`

7.1.2 Escalado robusto y eliminación de outliers

Se aplica `RobustScaler` para escalar las variables. A continuación se utiliza `EllipticEnvelope` con `contamination=0.10` para eliminar el 10 % de clientes más atípicos en términos multivariantes, dejando el dataset depurado con 3.905 clientes.

7.1.3 Reducción de dimensionalidad

Se aplica PCA con 3 componentes, que explican el **87,5 % de la varianza total** (PC1: 58,4 %; PC2: 18,1 %; PC3: 10,9 %). Esto permite visualización 3D sin pérdida significativa de información.

7.2 Descripción del modelo

DBSCAN (*Density-Based Spatial Clustering of Applications with Noise*) es un algoritmo de *clustering* basado en densidad, especialmente eficaz para identificar *clusters* de formas arbitrarias

y detectar puntos atípicos en grandes volúmenes de datos. A diferencia de K-Means, no requiere especificar el número de *clusters* a priori. En su lugar, utiliza dos parámetros clave:

- **eps** (ε): el radio máximo de vecindad que define la distancia de búsqueda entre puntos.
- **min_samples**: el número mínimo de puntos requeridos dentro del radio ε para que un punto se considere central (*core point*).

El objetivo es identificar grupos de clientes similares basándose en sus patrones de comportamiento de compra, permitiendo la detección simultánea de clientes atípicos.

7.3 Configuración del entrenamiento

Se realiza una búsqueda exhaustiva de parámetros óptimos. El rango de **eps** se calcula dinámicamente a partir del percentil de las distancias al k -vecino más cercano, obteniéndose un intervalo de **0,0312 a 0,2052**. Se exploran 100 combinaciones (**eps**=20 valores, **min_samples**=5 valores) y se evalúan con métricas múltiples (silueta, Davies-Bouldin, Calinski-Harabasz) combinadas con restricciones de equilibrio entre *clusters*.

```
def calcular_rango_eps(X, min_samples_ref=5):
    nbrs = NearestNeighbors(n_neighbors=min_samples_ref).fit(X)
    distancias, _ = nbrs.kneighbors(X)
    distancias_k = distancias[:, min_samples_ref - 1]
    eps_min = float(np.percentile(distancias_k, 10))
    eps_max = float(np.percentile(distancias_k, 70))
    return eps_min, eps_max, distancias_k
```

7.4 Modelo final y métricas

Tras la búsqueda, la configuración óptima seleccionada es **eps**=0.1411 y **min_samples**=15. El Cuadro 8 resume el modelo final.

Table 8: Resumen del modelo DBSCAN final

Métrica / parámetro	Valor
eps	0,1411
min_samples	15
<i>Clusters</i> encontrados	4
Puntos de ruido	1.927 (49,3 %)
Puntos asignados a <i>cluster</i>	1.978 (50,7 %)
Silueta	0,3386
Davies-Bouldin	0,5999
Calinski-Harabasz	1.012,36
Componentes PCA	3 (87,5 % varianza)

7.5 Análisis visual de los resultados

7.5.1 Gráfico K-distancias

El gráfico de k -distancias (Figura 12) muestra las distancias ordenadas de cada punto al k -ésimo vecino más cercano. La línea punteada roja indica el valor óptimo de **eps** (0,1411) detectado automáticamente, donde el gráfico presenta una inflexión clara que separa puntos densamente empaquetados de aquellos más aislados.

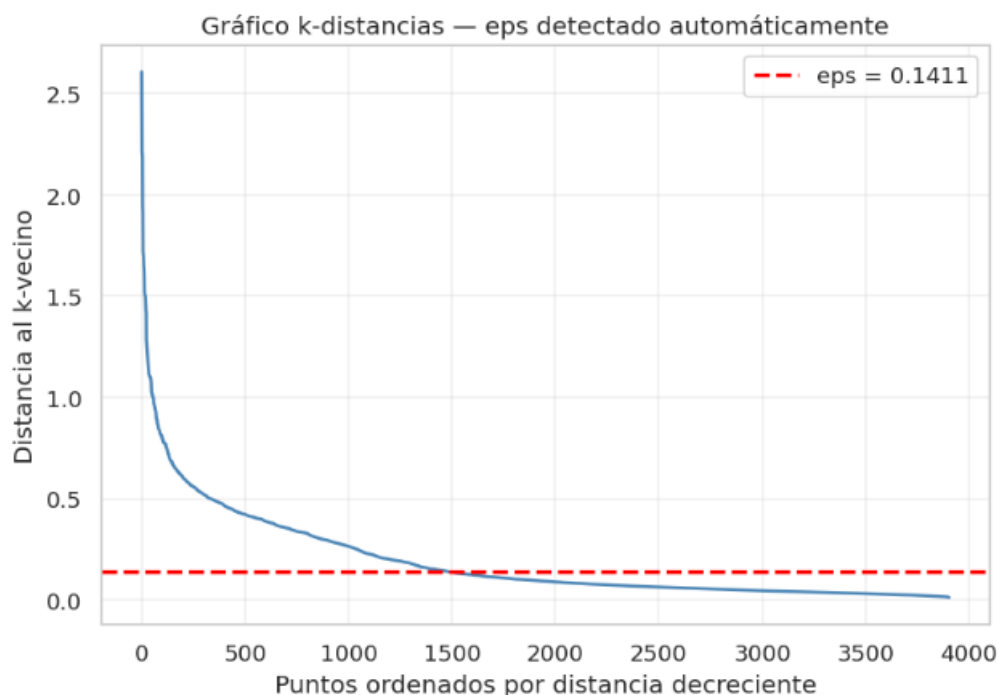


Figure 12: Gráfico k -distancias con eps óptimo detectado automáticamente.

7.5.2 Visualización 2D y 3D en espacio PCA

La proyección 2D (Figura 13) revela la distribución de clientes en los dos componentes principales que explican el 76,5 % de la varianza. Se observan claramente 4 *clusters* con patrones distintos, así como puntos aislados clasificados como ruido. La visualización 3D (Figura 14) incorpora la tercera componente y proporciona una perspectiva más completa de la separación.

7.5.3 Distribución de tamaños de clusters

La Figura 15 muestra que los *clusters* tienen tamaños desiguales: *Cluster* 0 (868 clientes), *Cluster* 1 (1.092 clientes), *Cluster* 2 (16 clientes) y *Cluster* 3 (2 clientes). Esta distribución es característica de DBSCAN, que identifica *clusters* de densidad variable.

7.5.4 Análisis Silhouette

El análisis de silueta por *cluster* (Figura 16) muestra la solidez de la asignación. La línea punteada roja (0,339) indica la media general.

7.5.5 Heatmap de perfiles por cluster

El *heatmap* (Figura 17) visualiza las características promedio de cada *cluster* normalizadas como z -scores. Los colores verdes indican valores altos y los rojos valores bajos.

7.6 Perfiles de clientes por cluster

Cluster 0 — Clientes Ocasionales (868 clientes).

- Total compras: 13,7 (bajo)
- Valor total: 250,45 € (bajo)
- Días como cliente: 1,03 (muy recientes)

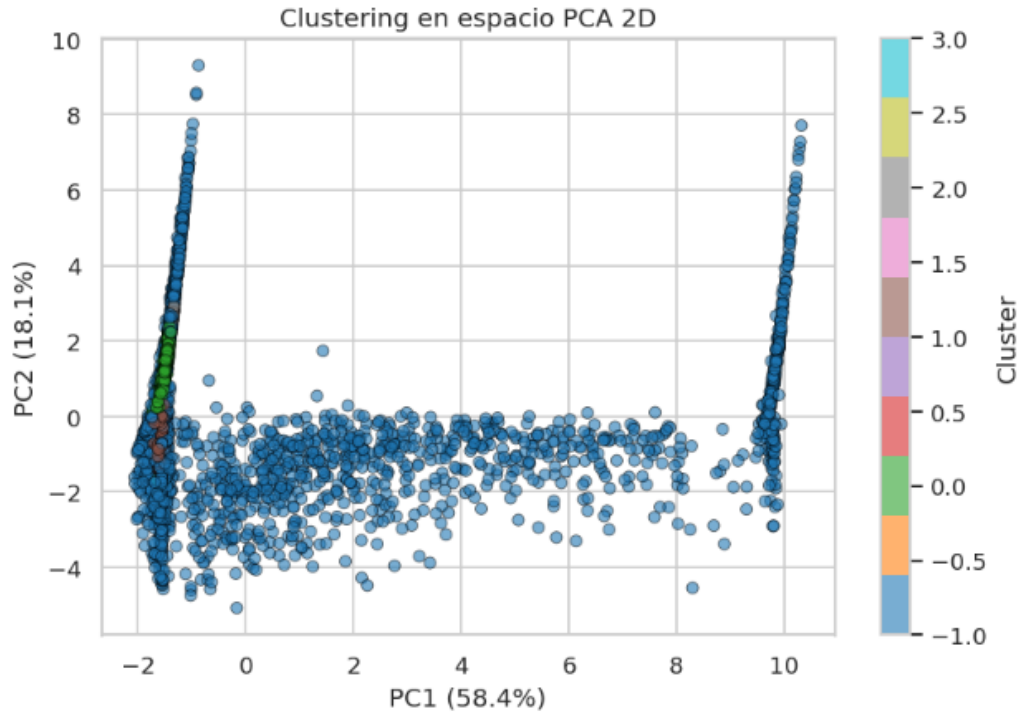


Figure 13: Clientes proyectados sobre las dos primeras componentes principales.

- **Perfil:** clientes nuevos o muy poco activos, posibles *leads* que aún no han realizado compras significativas.

Cluster 1 — Clientes VIP Activos (1.092 clientes).

- Total compras: 54,7 (muy alto)
- Valor total: 848,41 € (muy alto)
- Días como cliente: 167,8 (clientes establecidos)
- **Perfil:** clientes leales y de alto valor, son el corazón del negocio con compras frecuentes y gasto total elevado.

Cluster 2 — Clientes Estacionales (16 clientes).

- Total compras: 29,7 (medio)
- Valor medio por compra: 6,02 € (muy bajo)
- Frecuencia compra: 29,7 (alta)
- **Perfil:** clientes con compras muy frecuentes pero de bajo valor unitario; probablemente comparten características estacionales o de producto específico.

Cluster 3 — Clientes Premium Excepcionales (2 clientes).

- Total compras: 129,81 (extremadamente alto)
- Valor total: 2.329,16 € (máximo)
- Días como cliente: 241,96 (antigüedad máxima)

Clustering en espacio PCA 3D

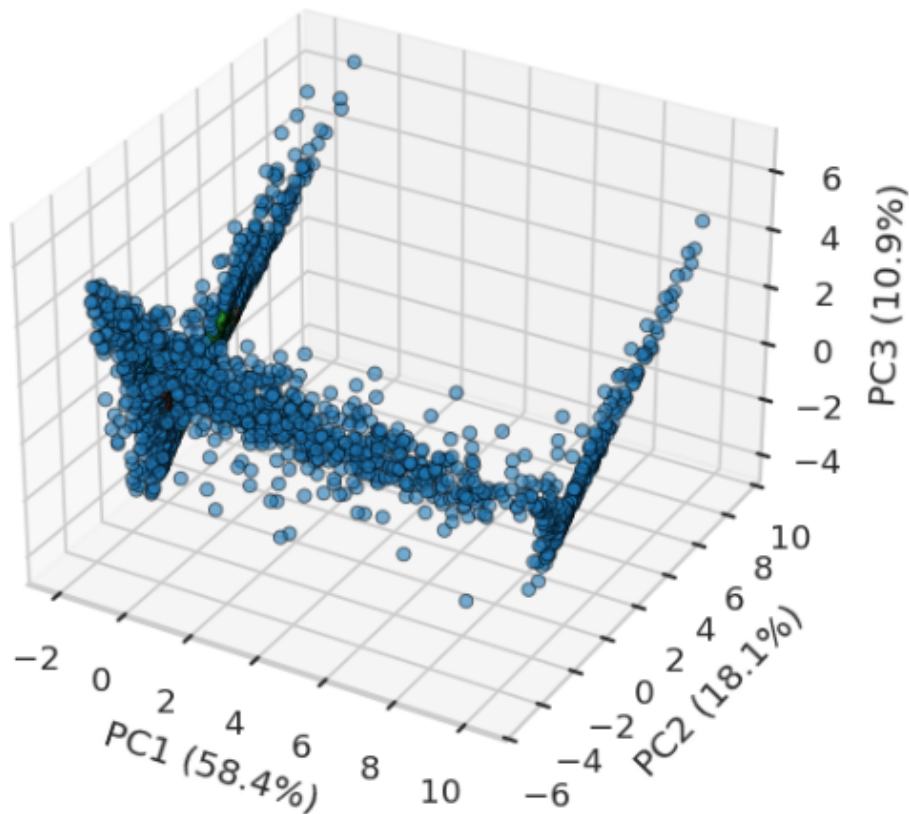


Figure 14: Clientes proyectados en las tres componentes principales del análisis PCA.

- **Perfil:** clientes de élite con comportamiento muy superior al resto; merecen estrategias de retención *premium* y atención personalizada.

7.7 Consideraciones sobre la validez del modelo

7.7.1 Fortalezas del modelo

- **Detección automática de anomalías:** DBSCAN identifica 1.927 clientes atípicos (49,3 %) sin necesidad de un paso adicional, que podrían representar fraude, errores de datos o comportamientos extraordinarios.
- **Sin necesidad de especificar k :** el algoritmo descubre automáticamente el número óptimo de *clusters* (4) basándose en la densidad de los datos.
- **Clusters de forma arbitraria:** puede identificar agrupaciones con formas no esféricas, especialmente útil para datos de comportamiento de clientes complejos.
- **Métricas sólidas:** Davies-Bouldin de 0,5999 indica buena separación; Calinski-Harabasz de 1.012,36 confirma alta densidad y separación entre grupos.

7.7.2 Limitaciones y consideraciones

- **Alto porcentaje de ruido (49,3 %):** más de la mitad de los clientes se clasificaron como atípicos. Esto puede indicar que el dataset tiene una distribución muy dispersa o que las

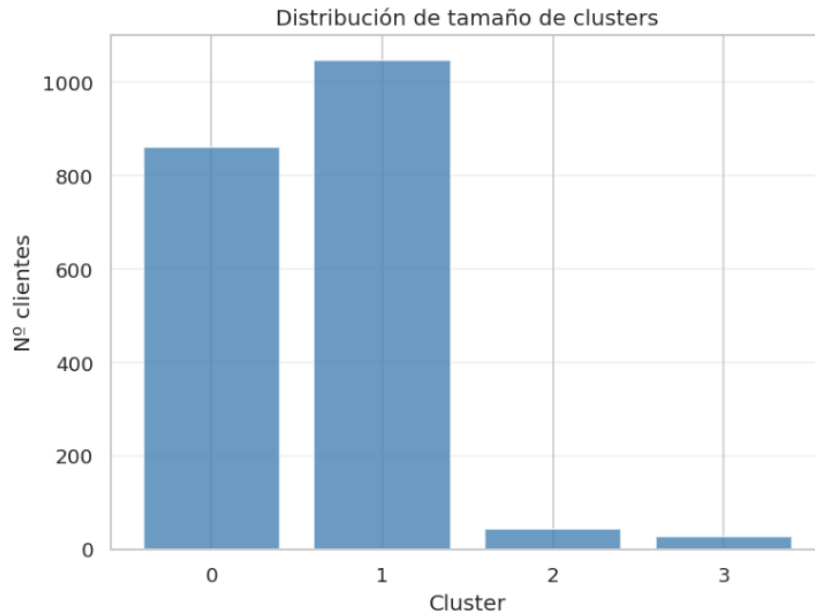


Figure 15: Distribución de tamaños de los *clusters* obtenidos.

características no capturan bien la verdadera estructura de comportamiento.

- **Clusters desbalanceados:** los *clusters* 2 y 3 son muy pequeños (16 y 2 clientes respectivamente), lo que limita la estabilidad de las conclusiones sobre estos segmentos.
- **Sensibilidad a parámetros:** DBSCAN es muy sensible a `eps` y `min_samples`; pequeños cambios pueden producir particiones muy distintas.
- **Silueta moderada (0,3386):** aunque aceptable, sugiere que algunos clientes están en la frontera entre *clusters* y podrían ser reclasificados con cambios menores en los parámetros.

8 Modelo HDBSCAN

8.1 Preprocesado específico para HDBSCAN

Sobre el dataset común generado en la Parte I se aplica un preprocesado pensado para preservar la información necesaria para detectar agrupaciones de densidad variable. Se seleccionan las 9 variables clave de comportamiento (las mismas que en DBSCAN), se imputan los valores faltantes con la mediana en cada columna y se aplica `StandardScaler` para escalar las variables al mismo rango. A diferencia de otros modelos, HDBSCAN no requiere una etapa explícita de eliminación de *outliers* previa, ya que el propio algoritmo identifica los puntos atípicos como ruido.

8.2 Descripción del modelo

HDBSCAN (*Hierarchical Density-Based Spatial Clustering of Applications with Noise*) es una extensión jerárquica de DBSCAN. A diferencia de DBSCAN, no requiere fijar un valor único de `eps`: construye una jerarquía de *clusters* a múltiples escalas de densidad y selecciona automáticamente la partición más estable. Esto le permite detectar *clusters* de densidades muy diferentes en un mismo dataset, algo que DBSCAN no consigue con un único parámetro.

Sus parámetros principales son:

- **`min_cluster_size`:** tamaño mínimo de un *cluster* válido. Influye en la granularidad de la segmentación.

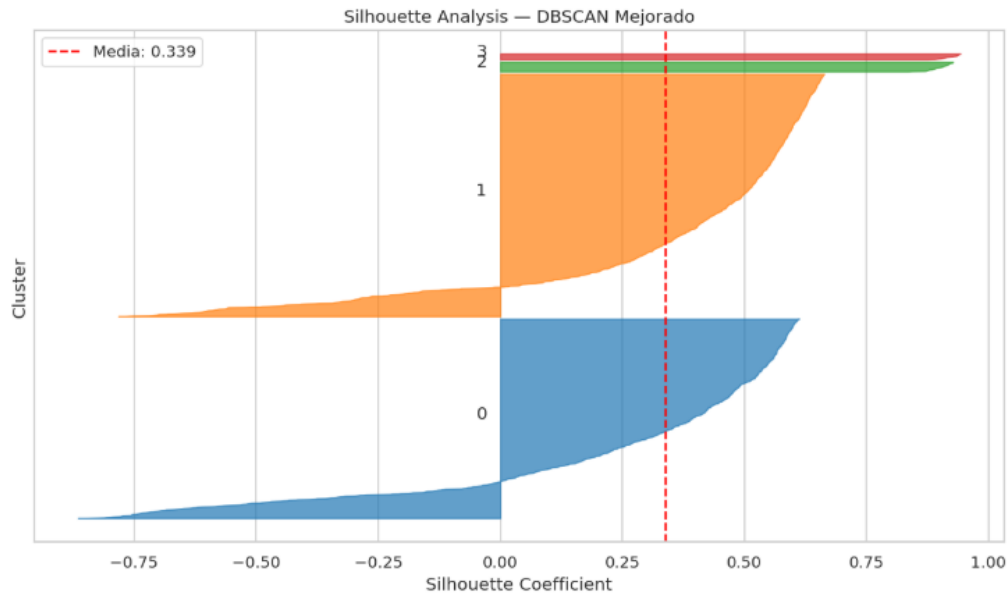


Figure 16: Análisis de silueta por *cluster*.

- **min_samples**: número mínimo de puntos requeridos para considerar una zona como densa.
- **cluster_selection_method**: estrategia de selección de *clusters* desde el árbol jerárquico (eom en este caso).

8.3 Configuración del entrenamiento

Se realiza una búsqueda exhaustiva de hiperparámetros explorando combinaciones de **min_cluster_size** $\in \{10, 15, 20, 25, 30, 40, 50\}$ y **min_samples** $\in \{1, 3, 5, 10\}$. Para cada combinación se calculan tres métricas (silueta, Davies-Bouldin, Calinski-Harabasz) más el equilibrio entre tamaños de *clusters*. La selección final se hace mediante una puntuación ponderada que combina las cuatro dimensiones, descartando configuraciones con más de 30 % de ruido o tamaños de *clusters* extremadamente desbalanceados.

```
clusterer = hdbscan.HDBSCAN(
    min_cluster_size=min_cluster_size,
    min_samples=min_samples,
    metric='euclidean',
    cluster_selection_method='eom'
)
etiquetas = clusterer.fit_predict(datos_escalados)
```

8.4 Modelo final

La configuración óptima identificada produce **8 clusters** sobre el dataset, con un porcentaje de ruido del **26,0 %** (1.127 clientes clasificados como atípicos), tal como puede observarse en las visualizaciones generadas (Figura 19).

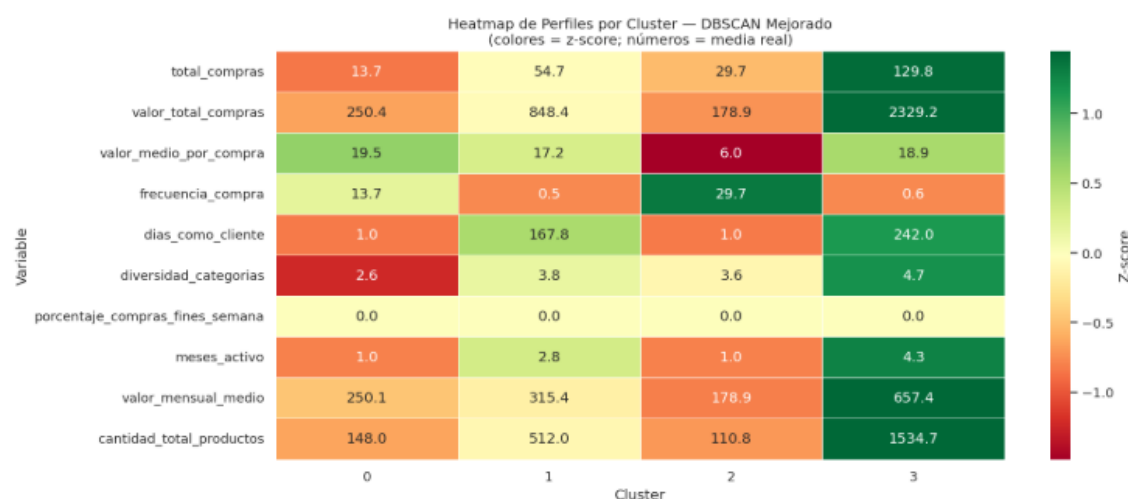


Figure 17: *Heatmap* de perfiles por *cluster*: colores codifican el *z-score*; números indican la media real.

8.5 Análisis visual de los resultados

8.5.1 Análisis de Silhouette

La Figura 18 muestra el coeficiente de silueta individual de cada cliente, agrupado por *cluster*. La línea discontinua roja indica el valor medio del análisis. Algunos *clusters* (0, 5, 6) presentan barras anchas y altas, mientras que otros (3, 4) muestran mayor variabilidad e incluso valores negativos en algunos puntos.

8.5.2 Distribución por segmento

La Figura 19 muestra la distribución de clientes en cada uno de los 8 *clusters* más el ruido. Los tamaños son notablemente desbalanceados, con un *cluster* mayoritario (1.360 clientes, 31,3 %), un grupo grande de ruido (1.127 clientes, 26,0 %) y varios *clusters* muy pequeños (de 68 a 255 clientes).

8.5.3 Visualización 2D en espacio PCA

La Figura 20 muestra los *clusters* proyectados sobre las dos primeras componentes principales (PC1: 36,7%; PC2: 22,8 %). Los centroides de los *clusters* aparecen marcados con "X" rojas y se concentran principalmente en una zona compacta del espacio, mientras los puntos de ruido se distribuyen ampliamente alrededor.

8.5.4 Distribución de distancias intra e inter-cluster

La Figura 21 muestra la distribución de distancias intra-*cluster* (azul, dentro del mismo grupo) e inter-*cluster* (rojo, entre grupos distintos). El ratio entre ambas distribuciones aparece anotado en la propia figura y permite valorar el grado de separación entre *clusters*.

8.5.5 Matriz de distancias entre centroides

La Figura 22 muestra la matriz de distancias entre los centroides de los 8 *clusters*. Permite identificar qué *clusters* son más similares entre sí (distancias bajas, color oscuro) y cuáles más diferentes (distancias altas, color claro).

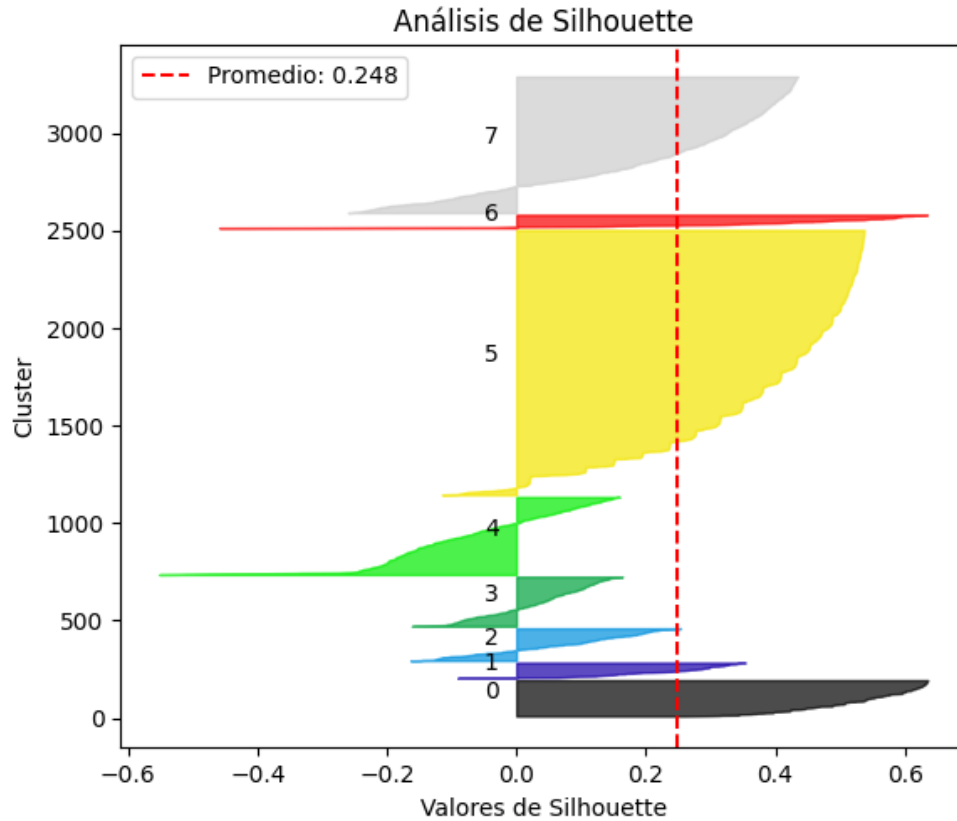


Figure 18: Coeficiente de silueta individual de cada cliente, agrupado por *cluster*.

8.5.6 Calidad por cluster (Boxplot)

La Figura 23 muestra mediante *boxplots* la distribución de coeficientes de silueta dentro de cada *cluster*. Permite identificar qué *clusters* tienen miembros más cohesionados.

8.5.7 Análisis de comportamiento de compra

Las Figuras 24, 25 y 26 muestran la distribución de los segmentos en distintas dimensiones de comportamiento: valor total vs frecuencia, ticket promedio por segmento y compras vs antigüedad.

8.6 Perfiles de clientes por cluster

El *pipeline* clasifica automáticamente cada *cluster* en un perfil de *marketing* a partir del comportamiento medio de sus miembros, comparado con los percentiles 25 y 75 del dataset completo. Los perfiles posibles son:

- **VIP:** alto valor y alta frecuencia. Programas de lealtad *premium*, acceso anticipado a productos.
- **Premium:** alto valor, baja frecuencia. Ofertas exclusivas, eventos VIP.
- **Frecuentes:** bajo valor, alta frecuencia. *Cross-selling*, descuentos por volumen.
- **Nuevos:** recién llegados, bajo valor. Ofertas de bienvenida, *onboarding* personalizado.
- **Inactivos:** bajo *engagement*. Campañas de reactivación.

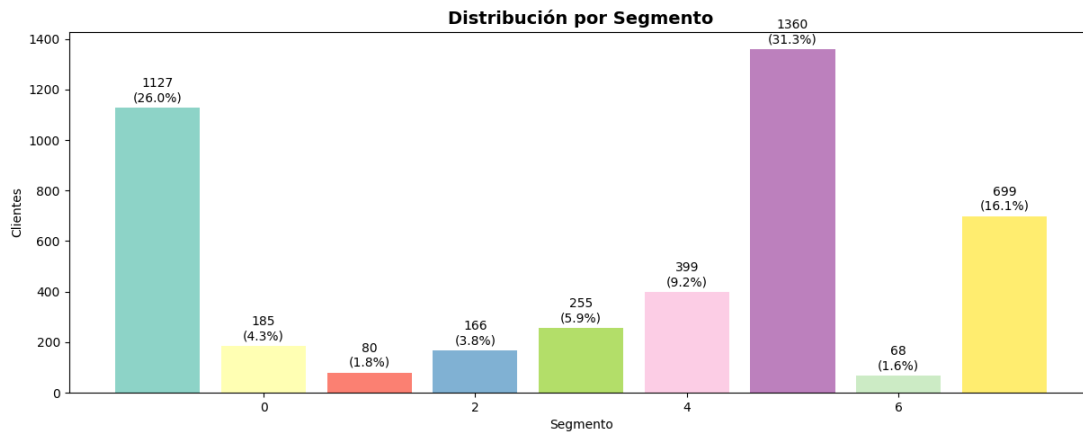


Figure 19: Distribución de clientes por segmento, incluyendo el ruido.

- **Regulares:** comportamiento estándar. *Marketing mix* equilibrado.

8.7 Consideraciones sobre la validez del modelo

8.7.1 Fortalezas del modelo

- **Detección automática de número de clusters:** HDBSCAN identifica 8 segmentos sin necesidad de fijar el valor a priori.
- **Detección automática de anomalías:** 1.127 clientes (26,0 %) son clasificados como ruido y separados del análisis principal.
- **Buena separación entre clusters:** las distribuciones de distancias intra e inter-*cluster* (Figura 21) confirman que los grupos están diferenciados en el espacio.
- **Capacidad de detectar clusters de densidad variable:** identifica simultáneamente grupos grandes y pequeños sin necesidad de elegir un único umbral de densidad.
- **Búsqueda automática de hiperparámetros:** el *pipeline* explora múltiples configuraciones y selecciona la más estable mediante una puntuación ponderada.

8.7.2 Limitaciones y consideraciones

- **Silueta moderada:** el valor medio observado se sitúa por debajo del umbral típico (0,25–0,5) que indicaría una separación clara, lo que sugiere solapamientos entre algunos *clusters*.
- **Tamaños muy desbalanceados:** con 8 *clusters* y un grupo mayoritario de 1.360 clientes (31,3 %) frente a algunos de menos de 100 clientes, el peso de los *clusters* pequeños en una estrategia comercial es limitado.
- **Porcentaje notable de ruido (26,0 %):** aunque dentro del límite establecido (30 %), implica que uno de cada cuatro clientes no se asigna a ningún segmento, lo que puede dificultar acciones específicas sobre ellos.
- **Mayor complejidad técnica:** HDBSCAN requiere una librería externa (`hdbscan`, fuera de `scikit-learn`) y la interpretación del árbol jerárquico es menos directa que la de modelos como K-Means.
- **Sensibilidad a la selección de variables:** como cualquier modelo basado en densidad, la calidad del resultado depende de que las 9 variables elegidas capturen efectivamente la estructura de comportamiento.

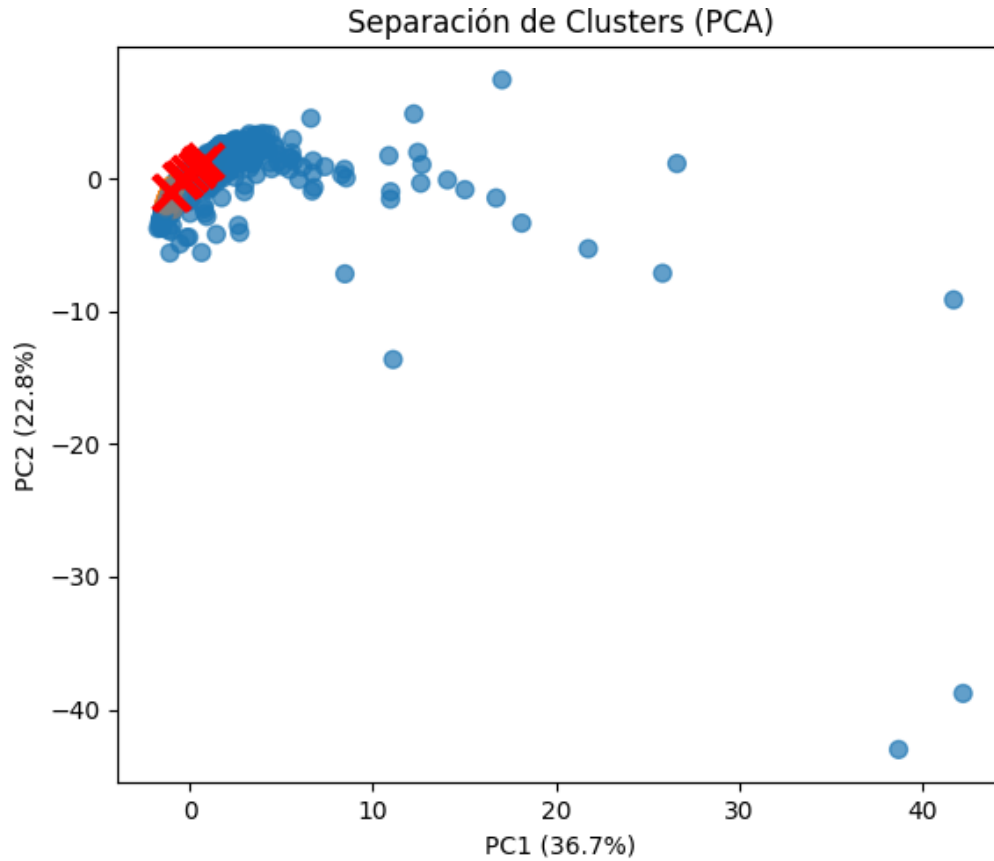


Figure 20: Separación de *clusters* en espacio PCA 2D.

9 Comparativa y modelo seleccionado

9.1 Tabla comparativa de los cuatro modelos

El Cuadro 9 resume los resultados de los cuatro modelos sobre el mismo dataset común generado en la Parte I. Los datos sirven como base para la decisión sobre el modelo a seleccionar como propuesta principal del proyecto.

Table 9: Comparativa de los cuatro modelos de *clustering*

Aspecto	K-Means	Jerárquico	DBSCAN	HDBSCAN
<i>Clusters</i> encontrados	2	2	4	8
Clientes utilizados	4.208	3.969	3.905	4.339
% eliminado preprocesado	3,0 %	8,5 %	10,0 %	—
% ruido (sin asignar)	—	—	49,3 %	26,0 %
Silueta	0,5257	0,7403	0,3386	0,248
Davies-Bouldin	—	0,7186	0,5999	—
Calinski-Harabasz	—	—	1.012,36	—
Tamaño <i>cluster</i> mín.	535 (12,7 %)	455 (11,5 %)	2 (0,1 %)	68 (1,6 %)
Tamaño <i>cluster</i> máx.	3.673 (87,3 %)	3.514 (88,5 %)	1.092 (28,0 %)	1.360 (31,3 %)
Detección de <i>outliers</i>	Externa (LOF)	Externa (P99+IF)	Nativa	Nativa
PCA aplicado	10 comp. (91,8 %)	3 comp. (84,1 %)	3 comp. (87,5 %)	No

Las casillas marcadas con “—” corresponden a métricas no calculadas o no aplicables al modelo en cuestión.

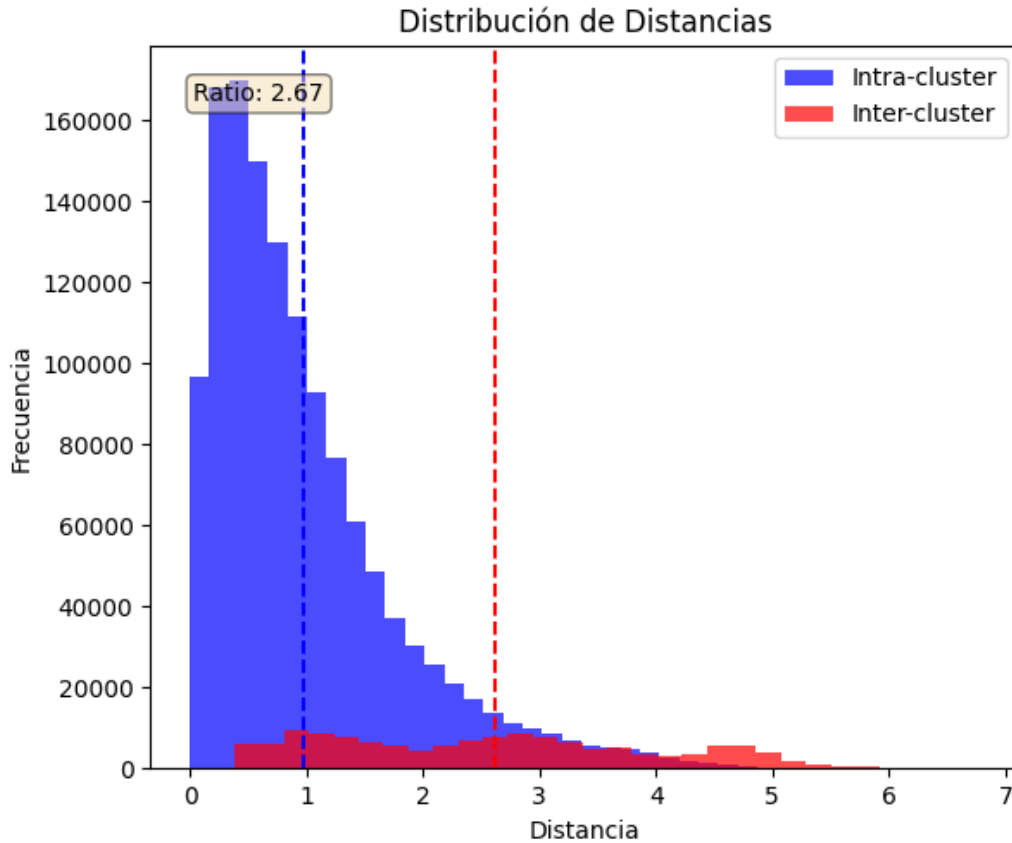


Figure 21: Distribución de distancias intra-*cluster* (azul) e inter-*cluster* (rojo).

9.2 Modelo seleccionado: K-Means

Tras analizar los cuatro modelos, el grupo selecciona como **modelo principal el K-Means** (versión con preprocesado mejorado mediante PCA + LOF), por las siguientes razones:

- **Convergencia entre criterios:** las tres métricas internas (silueta 0,5257, Calinski-Harabasz y Davies-Bouldin) señalan unánimemente $k = 2$ como configuración óptima, con una caída pronunciada y limpia de la silueta entre $k = 2$ (0,5257) y $k = 3$ (0,2627) que confirma la robustez de la elección.
- **Silueta dentro del rango “buena estructura” ($>0,5$),** suficiente para una segmentación operativa.
- **Conservación de datos:** utiliza el 97 % del dataset original (4.208 sobre 4.339), frente al 91 % del jerárquico o el 50 % efectivo de DBSCAN tras el ruido.
- **Simplicidad e interpretabilidad:** la naturaleza basada en centroides permite caracterizar cada *cluster* con un punto representativo, facilitando que los resultados se transformen directamente en campañas de *marketing* y acciones comerciales.
- **Aplicabilidad directa en CRM:** la asignación de cada cliente a un segmento concreto se exporta a CSV y es directamente integrable con plataformas de automatización comercial.
- **Rapidez y reproducibilidad:** el algoritmo es eficiente incluso en datasets grandes y, con `n_init=50` y `random_state=42`, los resultados son completamente reproducibles.

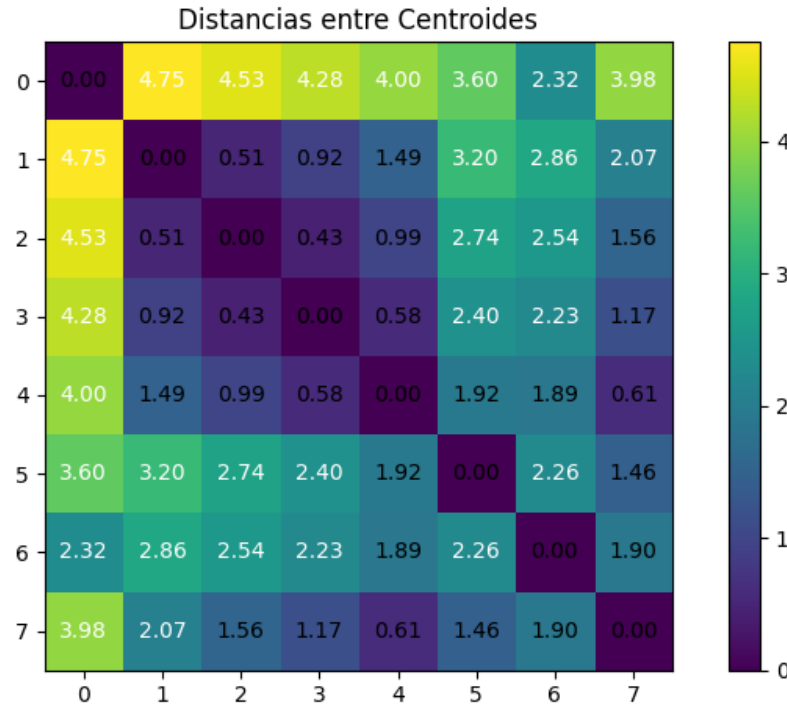


Figure 22: Matriz de distancias entre centroides de los 8 *clusters*.

9.3 Modelo de respaldo: Clustering Jerárquico

El **Clustering Jerárquico** queda documentado como **segunda opción válida**. Aunque ofrece una silueta superior (0,7403 vs 0,5257), su coste computacional es mayor ($O(n^2)$ en memoria) y la segmentación que produce es complementaria pero no esencialmente distinta de la del K-Means.

Lo más relevante es que **dos algoritmos completamente diferentes** (K-Means basado en centroides y Jerárquico basado en distancias jerárquicas) **coinciden en $k = 2$ con tamaños similares** (12 % / 88 %). Esta coincidencia es la mejor validación posible: la estructura encontrada es una propiedad real del dataset y no un artefacto de un método concreto. El dendrograma del jerárquico, además, aporta una herramienta de diagnóstico visual única que confirma visualmente que $K = 2$ corresponde al corte natural del dataset.

9.4 Modelos descartados: DBSCAN y HDBSCAN

Los modelos basados en densidad (DBSCAN y HDBSCAN) se descartan como propuesta principal por los siguientes motivos:

- **Elevado porcentaje de ruido:** DBSCAN clasifica el 49,3 % de los clientes como atípicos (casi la mitad del dataset queda sin asignar a ningún segmento) y HDBSCAN el 26,0 %. Para un proyecto de segmentación cuyo objetivo es accionar sobre toda la base de clientes, esto es una limitación crítica.
- **Presencia de clusters minúsculos sin valor de negocio:** DBSCAN identifica un *cluster* de solo 2 clientes y otro de 16. Aunque estadísticamente puedan mejorar las métricas, no son operativos para diseñar campañas comerciales.
- **Métricas inferiores:** silueta de 0,3386 (DBSCAN) y 0,248 (HDBSCAN), claramente por debajo de los modelos seleccionados.
- **Tamaños muy desbalanceados en HDBSCAN:** con 8 *clusters* y un grupo mayoritario de 1.360 clientes frente a algunos de menos de 100, el peso de los *clusters* pequeños en una

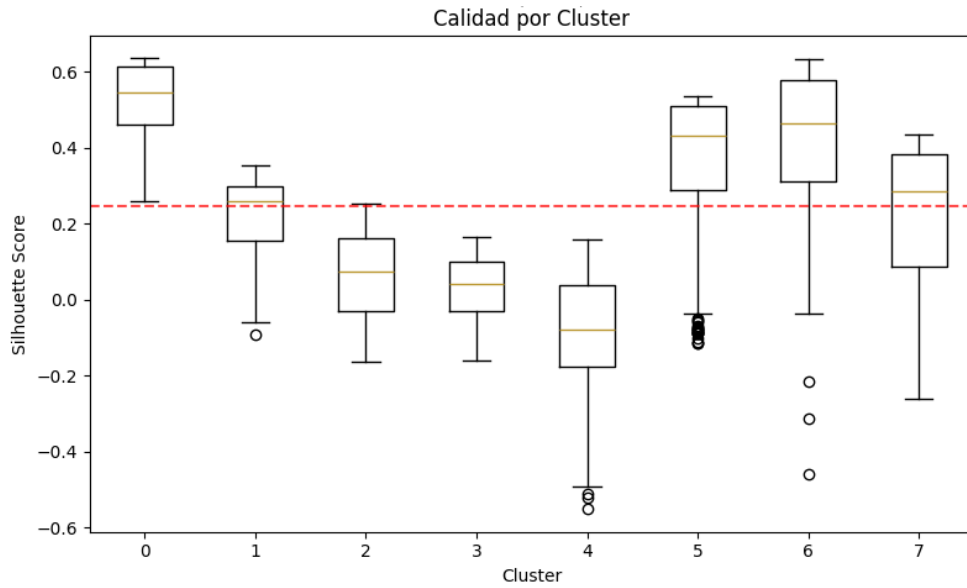


Figure 23: Distribución de coeficientes de silueta por *cluster* (*boxplot*).

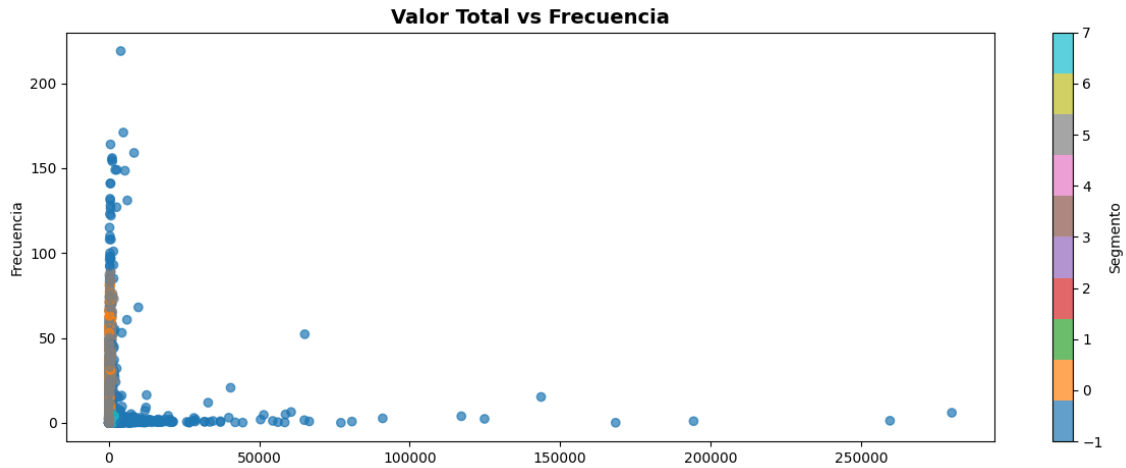


Figure 24: Valor total vs frecuencia de compra, coloreado por segmento.

estrategia comercial es limitado.

No obstante, el ejercicio de probarlos ha sido valioso porque demuestra que se han explorado enfoques alternativos basados en densidad y permite validar, por contraste, la elección final.

9.5 Conclusión final

La segmentación seleccionada divide la base de clientes en **dos grupos**, con tamaños aproximados del 12,7 % y 87,3 %. Esta estructura ha sido encontrada de forma independiente por dos algoritmos distintos (K-Means y Jerárquico), lo que aporta robustez metodológica. La propuesta final es por tanto utilizar K-Means como modelo de producción, con el Clustering Jerárquico como alternativa de validación cruzada.

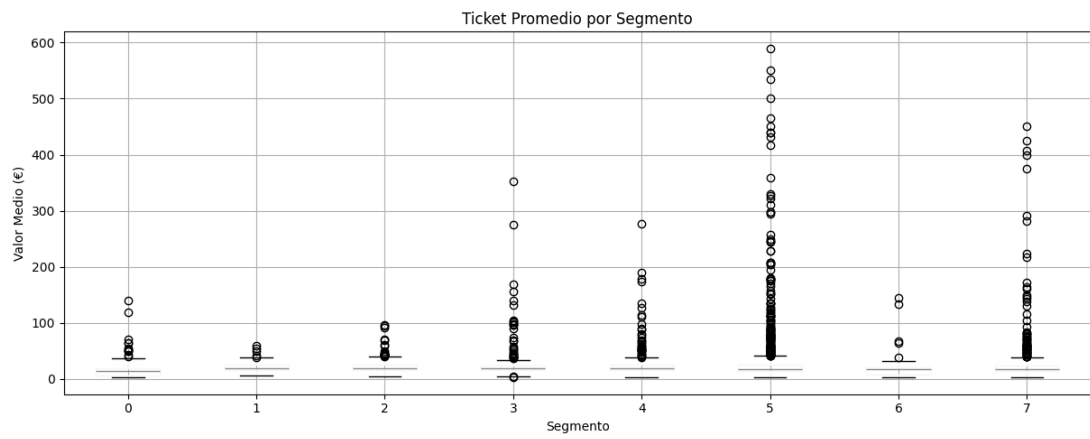


Figure 25: *Boxplot* del ticket promedio por segmento.

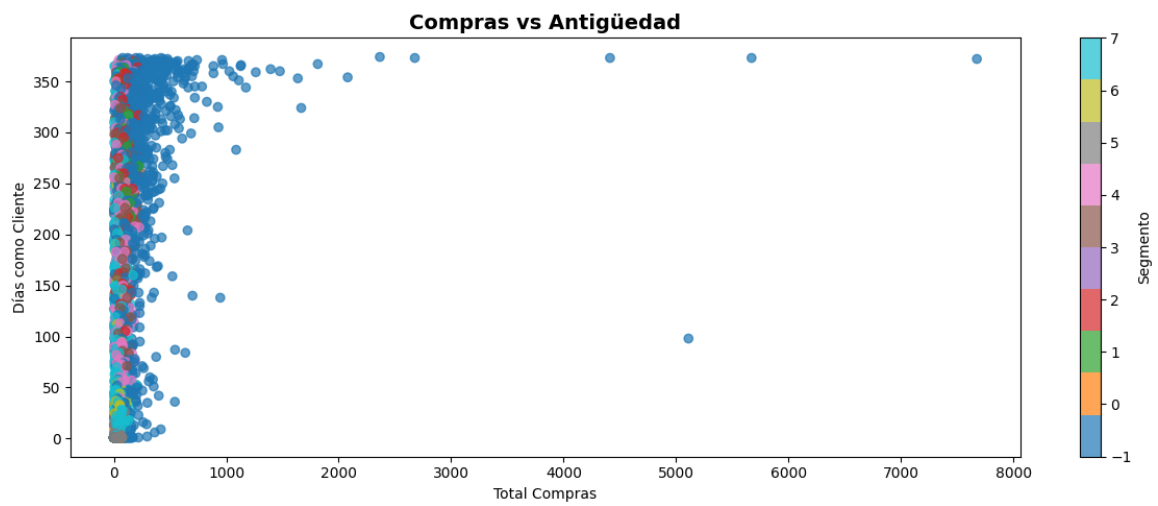


Figure 26: Total de compras vs antigüedad del cliente, coloreado por segmento.